

G큐브 외부 보드 (아두이노/큐브이노) 연결 매뉴얼

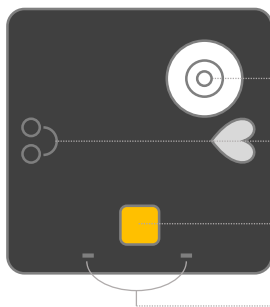
Open platform IoT robotics module



G-Cube

G큐브는 CPU, 모터, 배터리, 각종 센서가 내장된 IoT모듈로 G큐브 1개로부터 로봇을 만들 수 있고 여러 개를 블루투스로 연결하여 무한 확장하며 로보틱스, AI, SW, 사물인터넷, 빅데이터 프로젝트에 활용할 수 있는 오픈 플랫폼입니다.

GCube

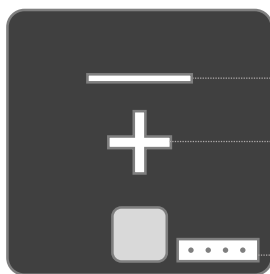


충전용 자석 단자 (상)

적외선 근접 센서

버튼

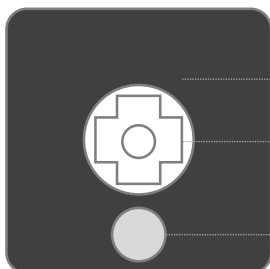
연결, 충전 표시등



LED 표시등

링크 홀

4핀 확장 포트
(GND, Vout, SCL, SDA)
(GND, Vout, TXD, RXD)



자이로센서, 부저

스텝 모터

도형 모양

GCube applications

G큐브

BLE5.0 CPU

배터리

스텝모터



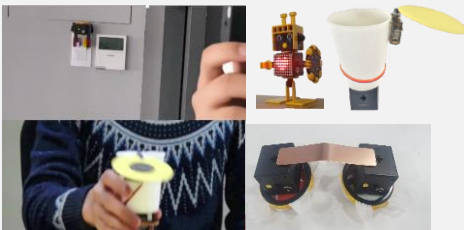
내장센서
[자이로/근접]
[버튼/부저]

외부센서포트

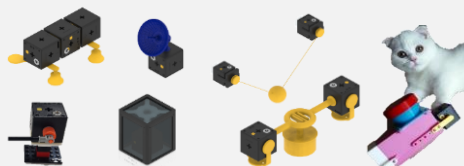
1 G큐브 + 링크 → 핑퐁 로봇



2 G큐브 + 센서 → IoT 네트워크



3 G큐브 + 3D 프린터 → 메이커



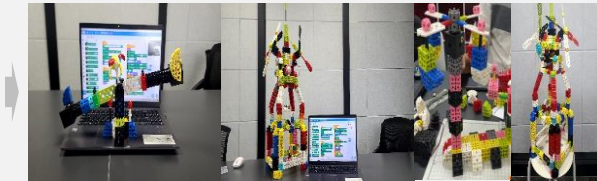
4 G큐브 + 레고 → 브릭큐브 로봇



5 G큐브 + 포디프레임 → 포디큐브로봇



6 G큐브 + 큐브웍스 → 큐브웍스 로봇



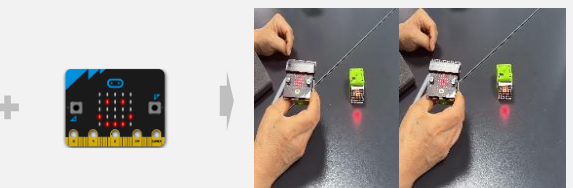
7 G큐브 + 케이넥스, MRT, 매직토이



8 G큐브 + 아두이노 → IoT 로봇틱스



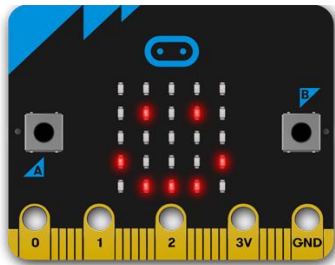
9 G큐브 + 마이크로비트 → SW 교육



Mission with Arduino

G큐브는 마이크로비트, 아두이노, 라즈베리파이 등의 보드와 Serial 통신으로 연결하여 보드를 사용하는 사용자가 G큐브를 활용한 로봇틱스와 사물인터넷 프로젝트를 쉽게 수행할 수 있도록 라이브러리 및 예제 프로그램을 제공합니다.

Micro:bit



Serial

G-Cube

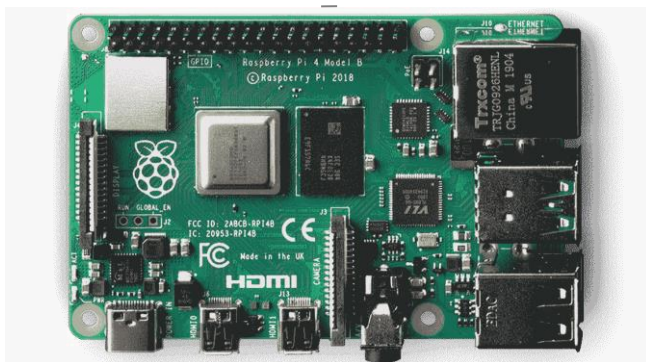


Serial

Serial



Arduino



Raspberry PI

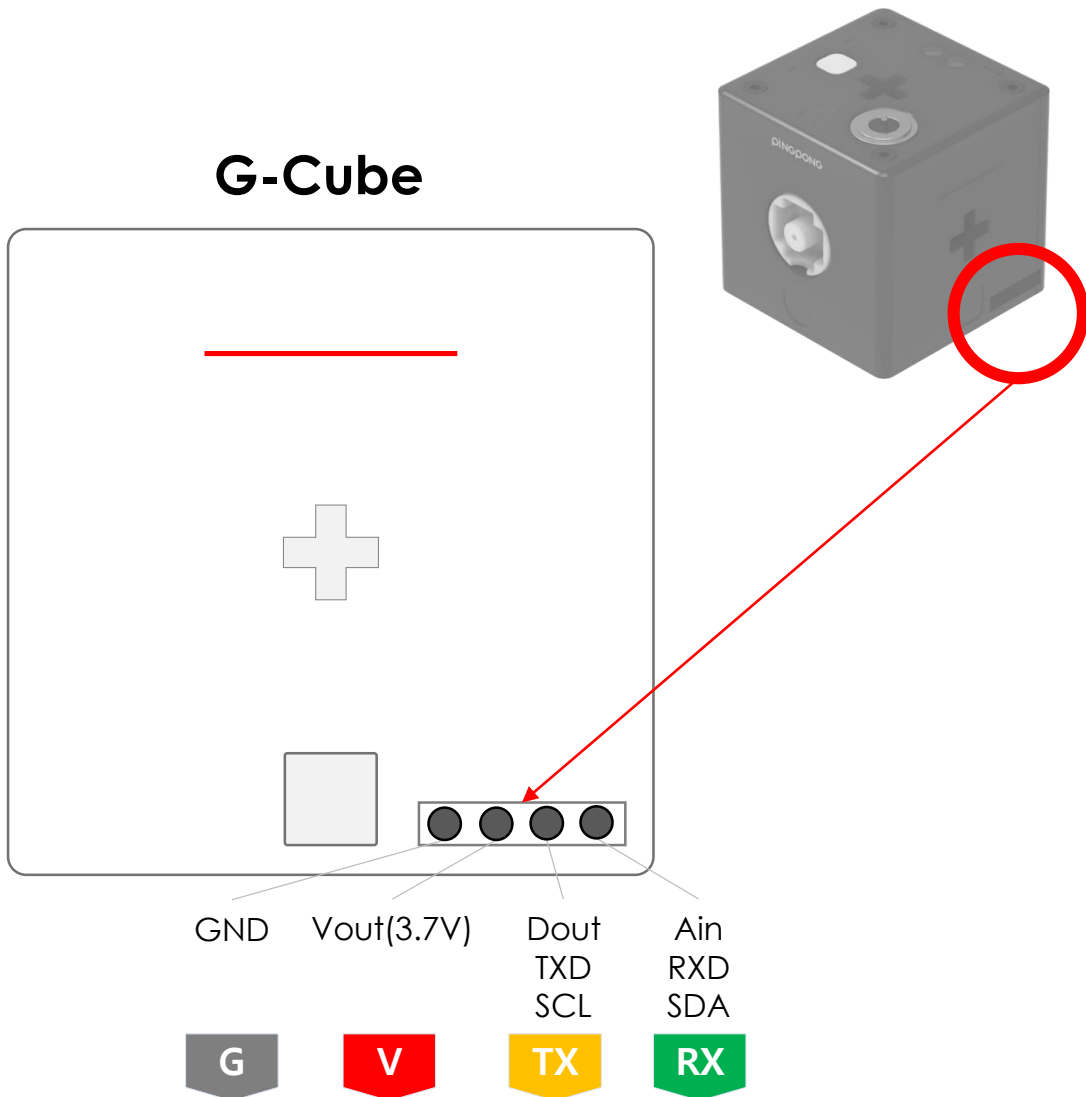
Pin Map (Serial connection)

G큐브와 외부 보드 (아두이노, 마이크로비트 등)를 연결할 때 G큐브 측면 하단에 아래와 외부 확장포트가 있습니다. 이 핀맵을 참조하여 보드에 점퍼 케이블로 연결해 줍니다.

Baudrate : 115200

Arduino pins : 2 (RX), 3 (TX), 5V, GND

Microbit pins : P0 (RX), P1 (TX), 3V, GND



- Vout : 큐브 내부의 배터리 전원을 밖으로 연결해 둔 포트로 3.7V ~ 4.2V 값을 가진다
- GND : 큐브 내부의 Ground.
- Dout/TXD/SCL : Digital 출력핀, 시리얼통신 TX 핀, I2C의 SCL 핀으로 사용한다.
- Ain/RXD/SDA : Analog **입력핀**, 시리얼통신 RX 핀, I2C의 SDA 핀으로 사용한다.

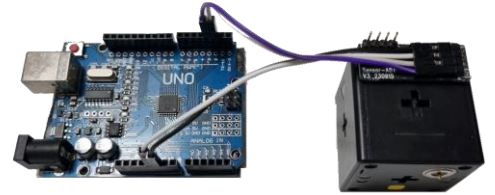
Connection of Arduino and GCube



Arduino UNO



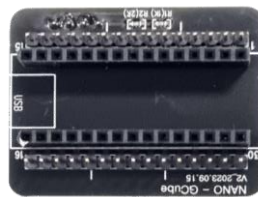
UNO Adaptor



UNO + GCube



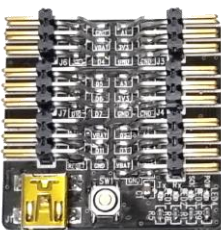
Arduino NANO



NANO Adaptor



NANO + GCube

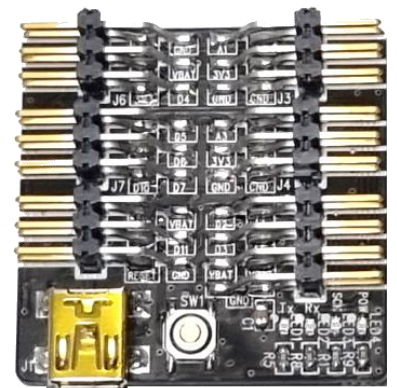
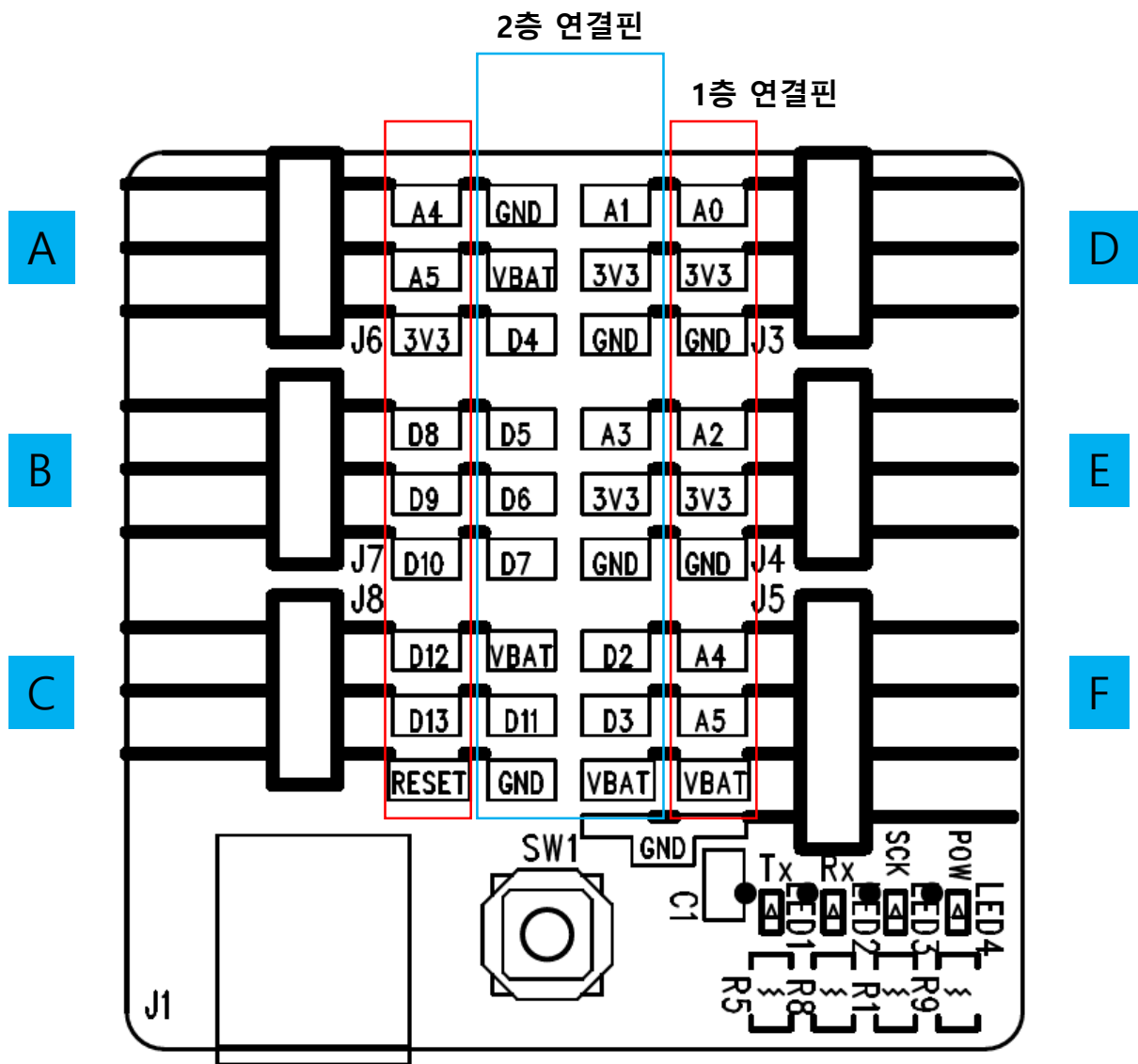


CUBEINO



CUBEINO + GCube

큐브이노는 로보라이즌에서 개발한 아두이노 우노 버전 보드입니다.



외부확장 연결핀 사용 가이드

[서보모터(SG90S, MG90S 등) 연결할 때]

- A 구역 2층 연결핀 사용

[아두이노용 아날로그센서 연결할 때]

- D구역 1~2층, E구역 1~2층에 꽂아서 A0~A4 신호 읽어 사용함
- 아날로그 A4, A5 신호는 A구역 1층 이용하여 사용함.

[디지털 신호 사용할 때]

- B구역 1~2층, C구역 1~2층, F구역 2층에서 D2~D13까지 사용함

[I2C 외부 기기 연결 사용할 때]

- F구역 1층핀 사용하여 I2C 외부 장치 연결

[시리얼통신 외부 기기 연결 사용할 때]

- F구역 2층핀 사용하여 시리얼통신 외부 장치 연결

[아두이노 부트로더 구울 때]

- C구역을 부트로더 라이팅 장치에 연결하여 사용

Connection of Arduino and GCube 0 ~

Arduino



Serial

- STEP 1 : G큐브-아두이노 예시 프로그램을 아두이노에 업로드
STEP 2 : 어댑터를 이용하여 G큐브 확장포트에 아두이노(우노/나노) 연결
* 연결하면 아두이노는 G큐브 전원 에 의해 자동 부팅됨
STEP 3 : 보드에 연결된 0번 G큐브 전원 ON 하면 보드와 자동연결
STEP 4 : 프로젝트에 필요한 (STEP 1의 아두이노 프로그램에 설정된 값) 큐브의 개수만큼 연결 되도록 다음 G큐브 전원버튼 2번 누름 (첫번째 누를 때 전원 ON, 두 번째 누를 때 0번 G큐브에 블루투스 연결됨)
STEP 5 : 필요한 큐브 연결이 끝나면 보드의 메인 미션 프로그램 수행됨



CUBE 0



CUBE 1



CUBE 2



- 보드가 연결된 0번 G큐브에 다른 큐브들을 연결하려면 연결될 큐브들이 모두 같은 그룹번호로 설정되어 있어야 한다.
- 그룹번호를 부여하는 방법은 Android/iOS 그룹위저드앱 (pingpong group wizard) 사용함 (YouTube 로보라이즌 채널 -> 교육동영상 참조)



CUBE 0



CUBE 1



CUBE 2



CUBE 3



CUBE 4



CUBE 5

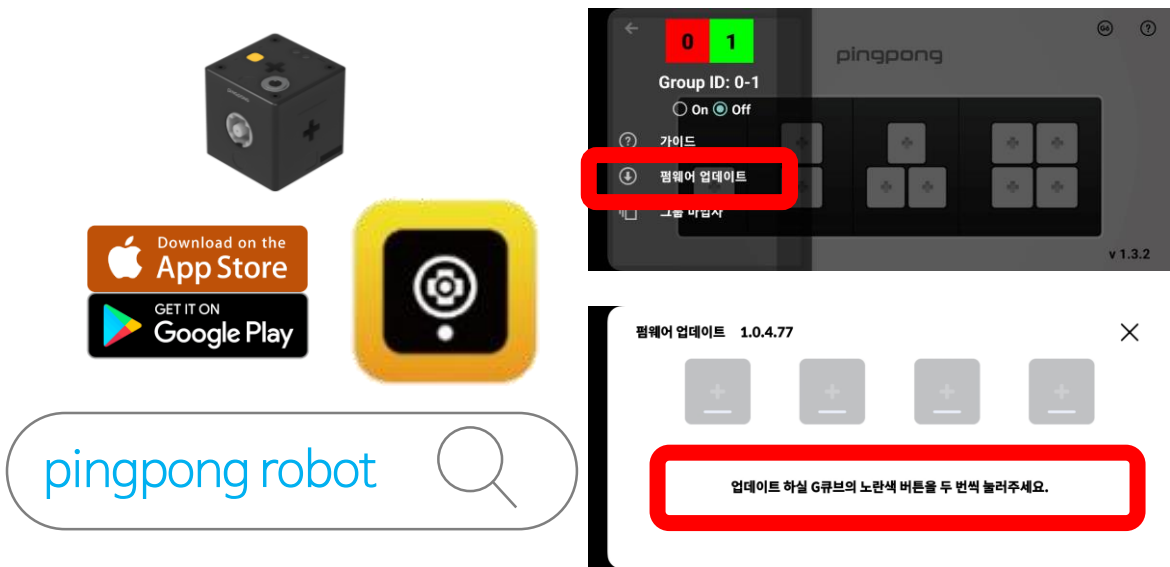
- 큐브 번호는 0번~7번까지 부여됨
- 0번큐브는 보드와 연결된 큐브이고 1~7번은 0번 큐브와 블루투스 연결된 큐브 (연결된 순서대로 번호부여)
- 0번 G큐브는 빨간색 LED가 켜지고 나머지 큐브들은 연결된 순서에 따라 Green, Blue, Cyan, Magenta, Yellow, Violet, Orange 등의 순서로 켜진다.

GCube 펌웨어 확인 및 그룹번호 맞추기

펌웨어 업데이트

마이크로비트/아두이노에 G큐브를 연결하여 사용하려면 G큐브의 펌웨어 버전이 77 이상의 버전이어야 합니다.

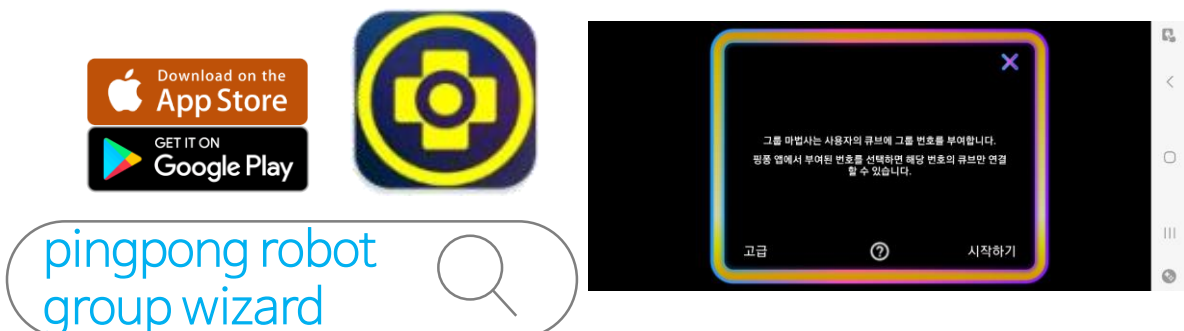
Google Play 또는 Apple store 에서 “pingpong robot” app을 설치하신 후 펌웨어 업데이트 화면에서 G큐브를 연결하여 펌웨어 업데이트를 진행합니다.



그룹번호 동일하게 지정하기

마이크로비트/아두이노에 G큐브를 연결하여 사용하려면 프로젝트에 사용할 G큐브의 그룹번호가 모두 동일하게 설정되어 있어야 합니다. 여러 명의 사람이 동시에 사용할 때 블루투스 연결 혼선을 막기 위해 동일한 그룹번호를 가진 G큐브들이 서로 연결됩니다.

- 그룹번호를 부여하는 방법은 Android/iOS 그룹위저드앱 (pingpong group wizard) 사용함 (YouTube 로보라이즌 채널 -> 교육동영상 참조)

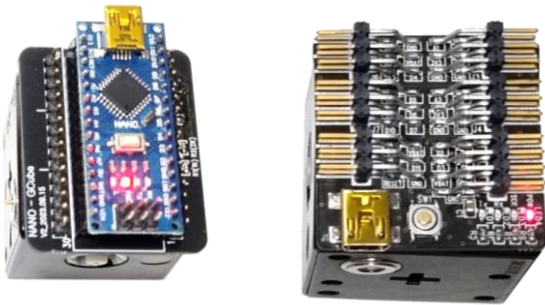


GCube 와 연결하고 실행하기

STEP 1 : 아두이노(나노/우노/큐브이노) (Gcube-Arduino) 예시 코드를 업로드 합니다



STEP 2 : 아두이노(우노/나노)를 어댑터에 꽂고(큐브이노는 직접연결) 어댑터를 G큐브 확장포트에 꽂아 연결합니다.



STEP 3 : 아두이노가 꽂혀 있는 G큐브 (0번큐브)의 노랑 버튼을 **한 번** 눌러줍니다.

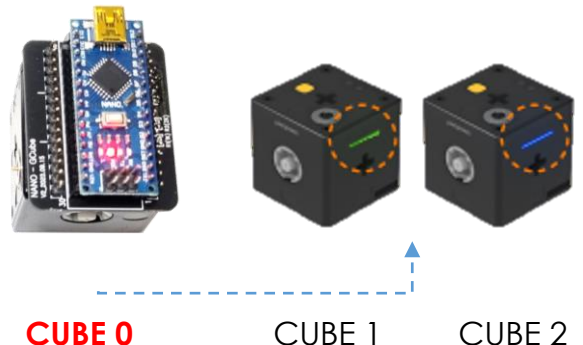
*아두이노에 예시코드가 정상적으로 업로드 된 경우 연결음이 나고 이 때 G큐브 옆면 LED도 빨강색으로 깜빡이며 켜집니다.

STEP 4 : 다른 G큐브의 노란색 버튼을 **두 번씩** 눌러줍니다. 연결되는 순서에 따라 초록색, 파랑색, 하늘색 ... 순서로 G큐브 옆면 LED가 켜집니다. 사용자가 아두이노 코드에서 설정한 숫자만큼의 큐브 (0번큐브 포함)가 모두 연결되면 아두이노의 메인 프로그램이 실행됩니다.

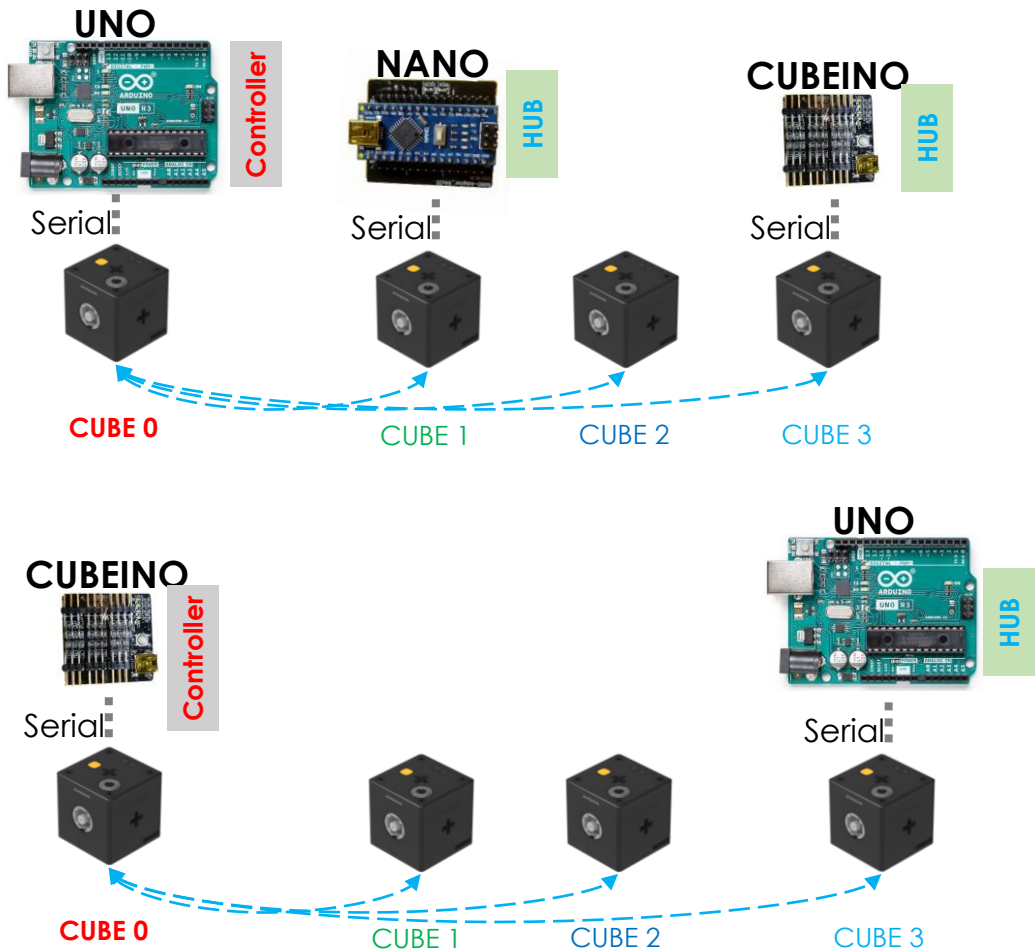
readCubeColorSensor | Arduino IDE 2.2.1

파일 편집 스케치 도구 도움말(H)

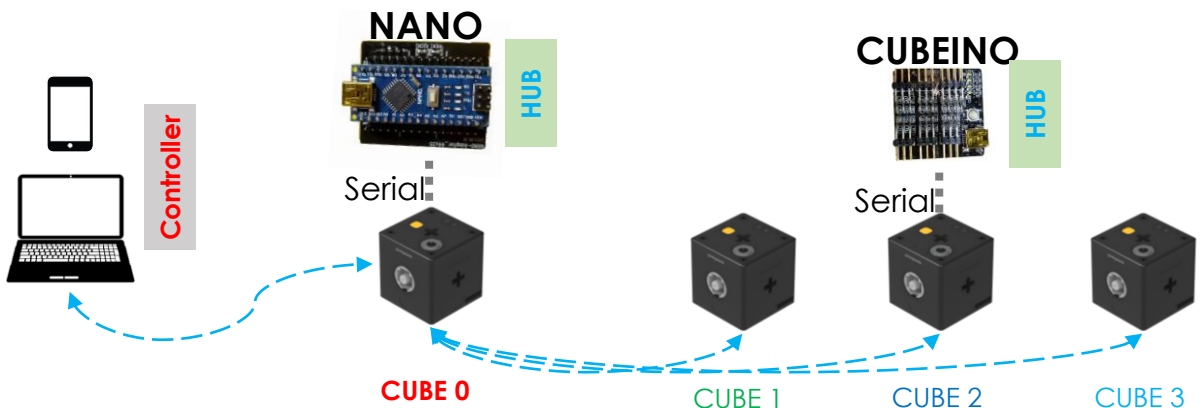
```
readCubeColorSensor.ino dotMatrix.h
1 #include <Servo.h>
2 #include "Cubeino.h"
3 #include "dotMatrix.h"
4 #include <SoftwareSerial.h>
5
6 #define GCUBE_TOTAL 3 // PC 시리얼로
7 SoftwareSerial mycubeserial (2, 3); //
```



Network configuration examples (1)

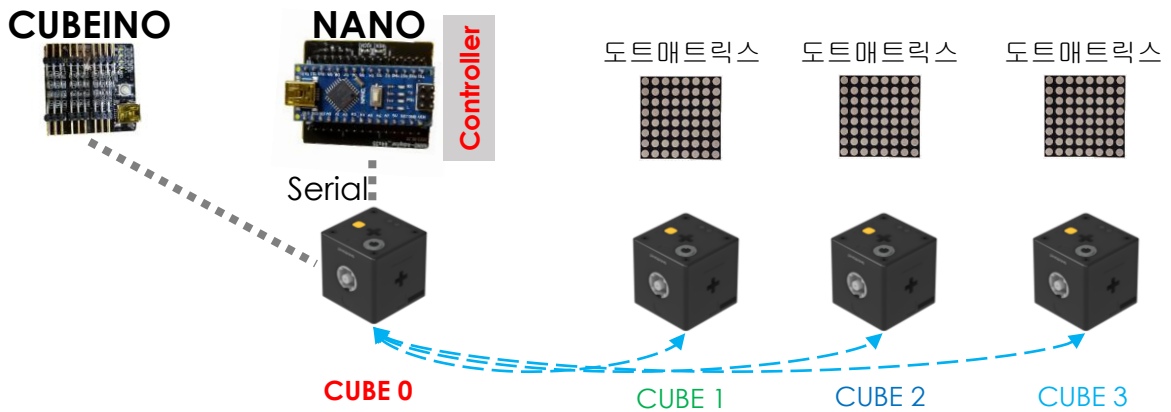


- 각 예시에서 왼쪽 아두이노/큐브이노에서 GCube 0~3의 모터 등을 제어할 수 있도록 Arduino IDE에서 코딩한다.
- 각 예시에서 왼쪽 아두이노/큐브이노에서 GCube 1~3의 각 센서 및 각 GCube에 연결되어 있는 아두이노 보드의 아날로그 값을 가져오거나 디지털 포트의 On/Off를 Arduino IDE에서 코딩한다.
- 위 그림에서 HUB 표시된 보드에는 라이브러리 예시 `sensorHub.ino` 프로그램이 업로드 되어 있어야 한다.

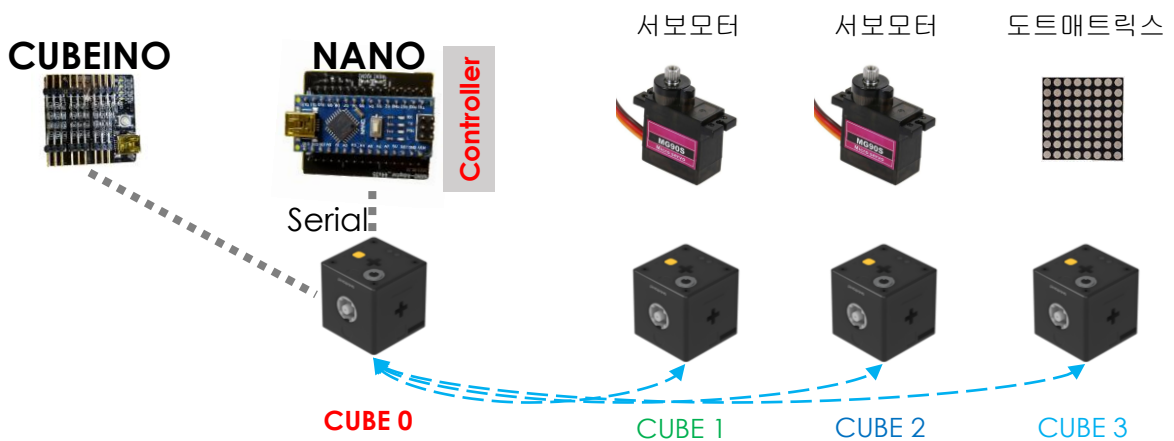


- 위 예시에서 왼쪽 스마트폰/노트북/크롬북 등에서 GCube 0~3를 블루투스와 연결하고 모터, 센서 등을 제어할 수 있도록 스크래치, 엔트리, 파이썬을 이용해 코딩한다.
- 위 예시에서 왼쪽 스마트폰/노트북/크롬북 등에서 GCube 0~3의 각 센서 및 각 GCube에 연결되어 있는 아두이노 보드의 아날로그 값을 가져오거나 디지털 포트를 제어하도록 스크래치, 엔트리, 파이썬을 이용해 코딩한다.

Network configuration examples (2)



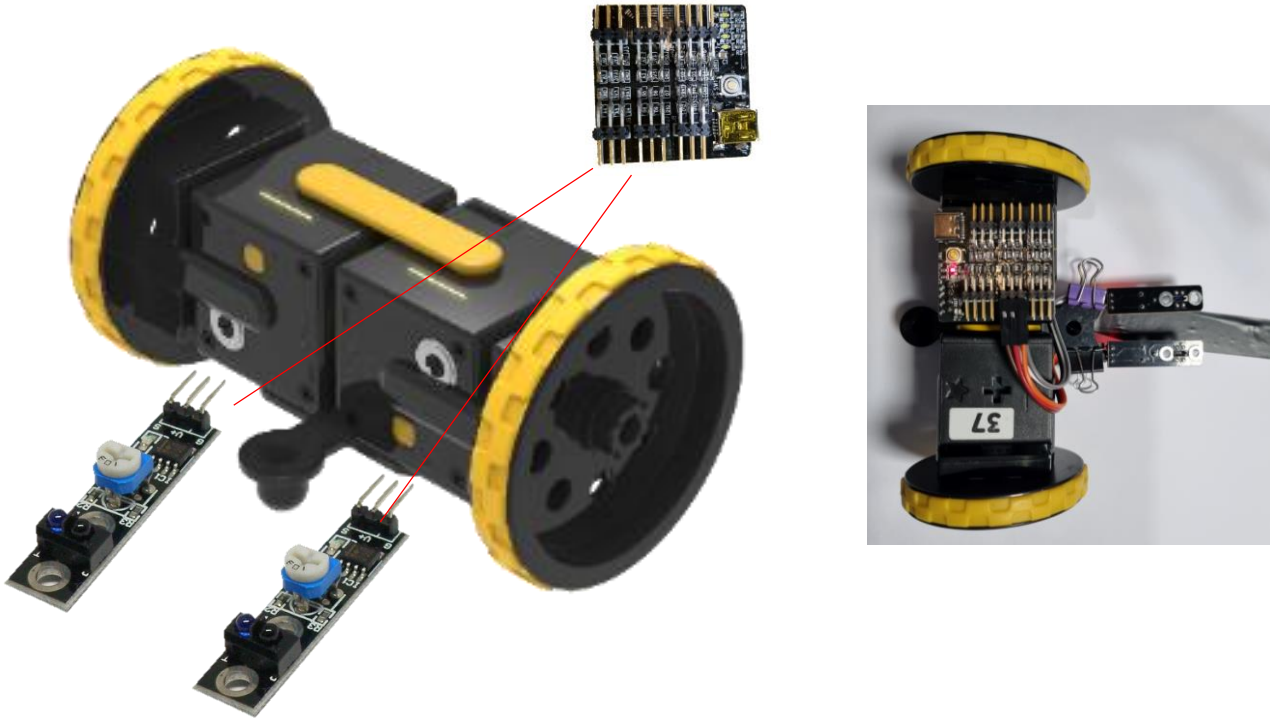
- GCube 0 에 나노 또는 큐브이노를 연결하고 GCube 0번을 켜다.
- GCube 1, 2, 3에 도트매트릭스를 켜둔 상태에서 전원 버튼을 두 번 눌러 GCube 0에 연결한다.
- 아두이노 라이브러리 예시 프로그램(artMatrix.ino)으로 아이콘/애니메이션을 매트릭스에 표현한다.



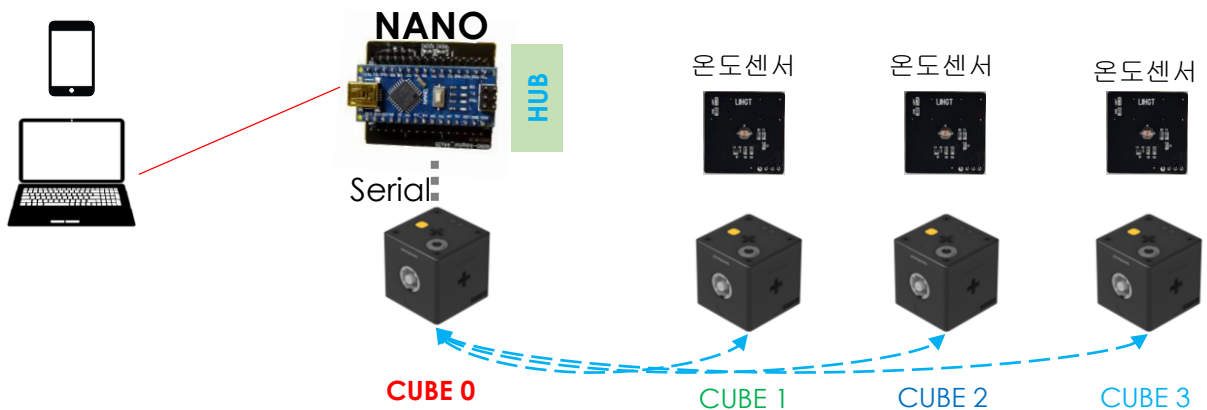
- GCube 0 에 나노 또는 큐브이노를 연결하고 GCube 0번을 켜다.
- GCube 1, 2 서보모터를 3에 도트매트릭스를 켜둔 상태에서 전원 버튼을 두 번 눌러 GCube 0에 연결한다.
- 아두이노 라이브러리 예시를 참조하여 서보모터와 도트매트릭스를 제어한다.

Application examples(1)

CUBEINO



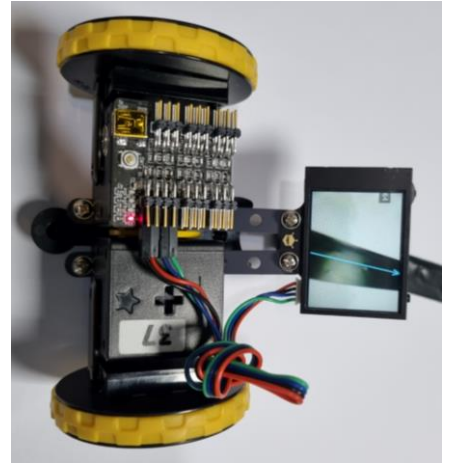
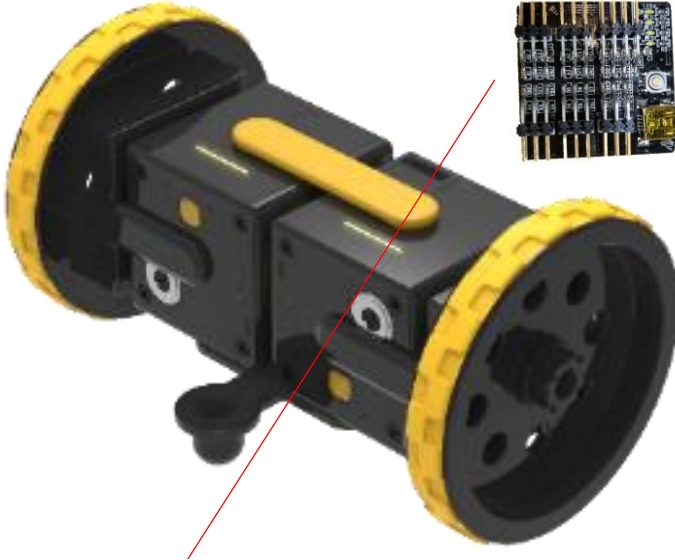
- 적외선 라인센서 2개를 큐브이노에 연결하고 큐브이노는 오토카 왼쪽 큐브에 꽂는다
- 큐브이노에 아두이노 IDE를 이용해 코딩을 하여 라인센서 값에 따라 GCube 0 (왼쪽)와 GCube 1 (오른쪽) 모터를 조정하여 라인트레이서를 만든다.



- 아두이노 NANO를 GCube 0에 연결을 하고 나머지 3개의 큐브에 온도센서를 연결한 뒤 버튼을 눌러 블루투스로 연결한다.
- 위 3개의 큐브를 실험장소 여러 곳에 배치하고 NANO에서 3개의 큐브에 있는 온도센서 값을 얻어와 저장한다.
- NANO는 컴퓨터에 USB케이블로 연결되어 Arduino IDE 시리얼 모니터에서 센서값을 표시한다.

Application examples(1)

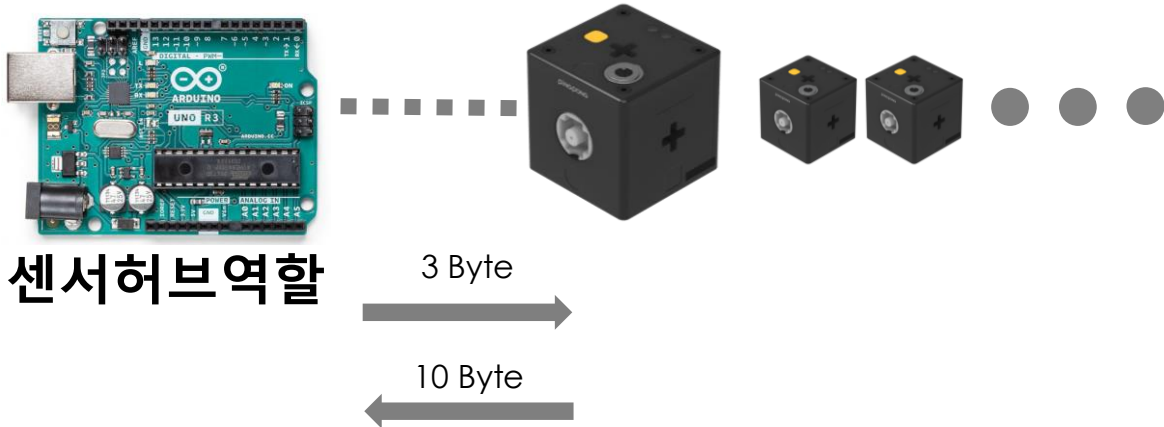
CUBEINO



HUSKY LENS

- 허스키렌즈를 큐브이노에 (I2C)연결하고 큐브이노는 오토카 왼쪽 큐브에 꽂는다.
- 허스키렌즈에서 라인(마스킹테이프 또는 종이위 마킹선 이용)을 학습시킨다.
- 큐브이노에 아두이노 IDE를 이용해 코딩을 하여 허스키라인센서 값에 따라 GCube 0 (왼쪽)와 GCube 1 (오른쪽) 모터를 조정하여 라인트레이서를 만든다.

Protocol – type A



G큐브 -> 보드 [3 Byte] : REQ

보드 -> G큐브 [10 Byte] : RCV (G큐브에서 물어본 것에 대한 대답), CMD (G큐브에 행동할 것을 명령함)

[G큐브에서 보드를 센서보드로 사용할 때]

*Zx는 임의의 숫자를 의미함, 보통 0을 입력하여 사용함

*IV 는 프로토콜 안정성 목적으로 명령어와 반대 순서 숫자 (예: REQ(0x12) -> IV(0x21)) 를 넣어줌

1. G큐브-Arduino 아날로그 값 주기적 전송

REQ(0x11) + period + bitFlag // G큐브에서 아두이노로 요구 명령 음 (한번만)

RCV(0x11) + IV + A3+ A2 + A1_hi + A1_lo + A0_hi + A0_lo + Z[~] // 주기적으로 아두이노 보드 아날로그값 읽어
서 G큐브로 보내줌

// bitFlag : 0000xxxx A3~A0 포트On/Off 비트 플래그

// period : 아날로그 읽는 주기 설정 (1~100, No response) // 1:100msec 100:10sec

2. G큐브-Arduino 디지털 포트 제어

REQ(0x1A) + PORT[H] + PORT[L] // 각 디지털 포트별 On/Off (Bit4:P4 ~ Bit13:P13)

3. 연결된 큐브 센서 값, 큐브-큐브이노(아두이노) 센서값 읽기

CMD(0xA0+CubeNumber) + IV + Z[~] // CubeNumber: GCube 0~7, GCube N 의 기울기 센서값 읽기 명령

REQ(0xA0+CubeNumber) + X + Y // GCube N 의 기울기 값 회신 (X, Y : -90~90), GCube N 의 기울기 센서값 읽
기 데이터

CMD(0xAA) + IV + CubeNumber + Z[~] // CubeNumber: GCube 0~7, GCube N 의 근접 센서값 읽기 명령

REQ(0xAA) + CubeNumber + P // GCube N 의 근접 센서값 읽기 데이터

CMD(0xAB) + IV + CubeNumber + Z[~] // CubeNumber: GCube 0~7, GCube N 의 버튼의 상태값(눌림/떼짐) 읽
기 명령

REQ(0xAB) + CubeNumber + B // GCube 버튼의 상태값 읽기 데이터

CMD(0xB0+CubeNumber) + IV + Z[~] //CubeNumber : GCube 0~7, GCube N 의 확장포트 센서값 읽기 명령

REQ(0xB0+CubeNumber) + Ext_hi + Ext_lo // GCube N 의 확장포트 센서값 읽기 데이터

//초음파(거리)센서 : 8bit(Ext_hi)

//소리,빛,볼륨,자기력 : 12bit (Ext_hi*16+Ext_lo) or 8bit(Ext_lo)

//온도센서 : 8bit+2bit(Ext_hi : 소숫점 위, Ext_lo : 소숫점 아래)

//칼라센서 : REQ Header(Red), Ext_hi(Green), Ext_lo(Blue)

CMD(0xE8+CubeNumber) + IV + Z[~] //CubeNumber : GCube 0~7, GCube N 의 확장포트-칼라센서값 읽기 명
령

REQ(Red) + Green + Blue // GCube N 의 확장포트-칼라센서값 읽기 데이터

CMD(0xD0+CubeNumber)+ IV + A[3~0] + Z[~] //CubeNumber : GCube 0~7, A3 ~ A0 : 3 ~ 0 //GCube N의 Ax
값 읽기 명령

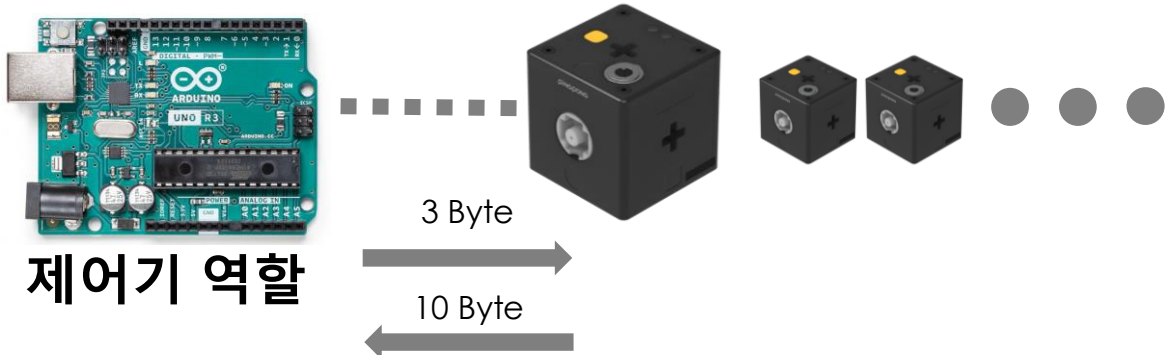
REQ(0xD0+CubeNumber) + A_hi + A_lo // 12 bit 이므로 Value = A_hi * 256 + A_lo //GCube N의 Ax 값 읽기 결
과

*A3, A2, A1의 경우는 8bit(1바이트) 분해능 이므로 A_lo로 데이터 보내고 A_hi = 0을 넣어줌..

CMD(0xD8) + IV + CubeNumber + Z[~] //CubeNumber 의 아날로그 센서 읽기 시작 모드 전송

CMD(0xD9) + IV + CubeNumber + PORT[H] + PORT[L] + Z[~] // 특정 큐브-아두이노의 디지털 포트 제어 명령

Protocol – type B(1/2)



[보드에서 G큐브를 제어하는 프로젝트로 사용할 때]

*Zx는 임의의 숫자를 의미함, 보통 0을 입력하여 사용함

*IV는 프로토콜 안정성 목적으로 명령어와 반대 순서 숫자 (예: REQ(0x12) -> IV(0x21))를 넣어줌

1. 보드 ID 확인

REQ(0x10) + Z1 + Z0

RCV(0x10) + IV + VID1 + VID0 + BID1 + BID0 + Z[~] (Vendor ID 2Byte + Board ID 2 Byte)

2. 연결된 큐브 개수 확인 및 연결완료 알려줌

CMD(0x21) + IV + Z[~] // STAR 큐브에 연결된 큐브 개수 확인해 줄것을 큐브에 요청

REQ(0x21) + Z[1] + N_CUBE (Number of Cube) // STAR 큐브에서 연결된 큐브 회신 함

CMD(0x2A) + IV + TOTAL_CUBE + Z[~] // STAR 큐브에 모든 큐브가 연결완료됨을 알려줌

3. 모터 회전 (스피드, 회전각도)

CMD(0x30) + IV + CubeNum + Speed + Z[~] // Set speed of a Cube (Speed : -100~100) //특정 번호 큐브 모터의 속도명령 전송

CMD(0x31) + IV + 0 + 0 + 0 + 0 + 0 + 0 + s1 + s0 // Set speed of Cube 0 ~ 1 (sx : -100~100) (0~1)큐브까지의 속도명령 전송

CMD(0x32) + IV + 0 + 0 + 0 + 0 + 0 + 0 + s2 + s1 + s0 // Set speed of Cube 0 ~ 2 (sx : -100~100) (0~2)큐브까지의 속도명령 전송

CMD(0x33) + IV + 0 + 0 + 0 + 0 + 0 + 0 + s3 + s2 + s1 + s0 // Set speed of Cube 0 ~ 3 (sx : -100~100) (0~3)큐브까지의 속도명령 전송

CMD(0x34) + IV + 0 + 0 + 0 + 0 + 0 + 0 + s4 + s3 + s2 + s1 + s0 // Set speed of Cube 0 ~ 4 (sx : -100~100) (0~4)큐브까지의 속도명령 전송

CMD(0x35) + IV + 0 + 0 + 0 + 0 + 0 + 0 + s5 + s4 + s3 + s2 + s1 + s0 // Set speed of Cube 0 ~ 5 (sx : -100~100) (0~5)큐브까지의 속도명령 전송

CMD(0x36) + IV + 0 + 0 + 0 + 0 + 0 + 0 + s6 + s5 + s4 + s3 + s2 + s1 + s0 // Set speed of Cube 0 ~ 6 (sx : -100~100) (0~6)큐브까지의 속도명령 전송

CMD(0x37) + IV + 0 + 0 + 0 + 0 + 0 + 0 + s6 + s5 + s4 + s3 + s2 + s1 + s0 // Set speed of Cube 0 ~ 7 (sx : -100~100) (0~7)큐브까지의 속도명령 전송

CMD(0x38) + IV + CubeNum + rxh + rxl + Z[~] // Set angle of a Cube (rxh/rxl : -10000~10000) //특정 번호 큐브 모터의 회전각도 명령 전송

CMD(0x39) + IV + 0 + 0 + 0 + 0 + 0 + 0 + r1h + r1l + r0h + r0l // Set angle of Cube 0 ~ 1 (rxh/rxl : -10000~10000) //0~1번 큐브 모터의 회전각도 명령 전송

CMD(0x3A) + IV + 0 + 0 + 0 + 0 + 0 + 0 + r2h + r2l + r1h + r1l + r0h + r0l // Set angle of Cube 0 ~ 2 (rxh/rxl : -10000~10000) //0~2번 큐브 모터의 회전각도 명령 전송

CMD(0x3B) + IV + 0 + 0 + 0 + 0 + 0 + 0 + r3h + r3l + r2h + r2l + r1h + r1l + r0h + r0l // Set angle of Cube 0 ~ 3 (rxh/rxl : -10000~10000) //0~3번 큐브 모터의 회전각도 명령 전송

CMD(0x3C) + IV + 0 + 0 + 0 + 0 + 0 + 0 + r5h + r5l + r4h + r4l // Set angle of Cube 4 ~ 5 (rxh/rxl : -10000~10000) //4~5번 큐브 모터의 회전각도 명령 전송

CMD(0x3D) + IV + 0 + 0 + 0 + 0 + 0 + 0 + r6h + r6l + r5h + r5l + r4h + r4l // Set angle of Cube 4 ~ 6 (rxh/rxl : -10000~10000) //4~6번 큐브 모터의 회전각도 명령 전송

CMD(0x3E) + IV + 0 + 0 + 0 + 0 + 0 + 0 + r7h + r7l + r6h + r6l + r5h + r5l + r4h + r4l // Set angle of Cube 4 ~ 7 (rxh/rxl : -10000~10000) //4~7번 큐브 모터의 회전각도 명령 전송

Protocol – type B(2/2)

4. 서보 모터 각도 제어

CMD(0x40) + IV + CubeNum + a0 + Z[~] // Set servo angle of a Cube : (ax : 0~180) //특정 번호 큐브에 장착된 서보모터 각도 제어

CMD(0x41) + IV + 0 + 0 + 0 + 0 + 0 + 0 + a1 + a0 // Set speed of Cube 0 ~ 1 (ax : 0~180) (0~1) 큐브_서보모터의 속도명령전송

CMD(0x42) + IV + 0 + 0 + 0 + 0 + 0 + 0 + a2 + a1 + a0 // Set speed of Cube 0 ~ 2 (ax : 0~180) (0~2) 큐브_서보모터의 각도명령전송

CMD(0x43) + IV + 0 + 0 + 0 + 0 + 0 + a3 + a2 + a1 + a0 // Set speed of Cube 0 ~ 3 (ax : 0~180) (0~3) 큐브_서보모터의 각도명령전송

CMD(0x44) + IV + 0 + 0 + 0 + 0 + a4 + a3 + a2 + a1 + a0 // Set speed of Cube 0 ~ 4 (ax : 0~180) (0~4) 큐브_서보모터의 각도명령전송

CMD(0x45) + IV + 0 + 0 + a5 + a4 + a3 + a2 + a1 + a0 // Set speed of Cube 0 ~ 5 (ax : 0~180) (0~5) 큐브_서보모터의 각도명령전송

CMD(0x46) + IV + 0 + a6 + a5 + a4 + a3 + a2 + a1 + a0 // Set speed of Cube 0 ~ 6 (ax : 0~180) (0~6) 큐브_서보모터의 각도명령전송

CMD(0x47) + IV + a7 + a6 + a5 + a4 + a3 + a2 + a1 + a0 // Set speed of Cube 0 ~ 7 (ax : 0~180) (0~7) 큐브_서보모터의 각도명령전송

* 0번 큐브 (STAR)에 서보모터 명령어가 주어지면 큐브이노에 붙어 있는 서보모터 (D3 PWM이용)를 제어함 (핑퐁 모노용)

5. 도트매트릭스

CMD(0x51) + IV + r7 + r6 + r5 + r4 + r3 + r2 + r1 + r0 (DotMatrix data of Cube 1) //1번 큐브에 장착된 도트매트릭스 화면

CMD(0x52) + IV + r7 + r6 + r5 + r4 + r3 + r2 + r1 + r0 (DotMatrix data of Cube 2) //2번 큐브에 장착된 도트매트릭스 화면

CMD(0x53) + IV + r7 + r6 + r5 + r4 + r3 + r2 + r1 + r0 (DotMatrix data of Cube 3) //3번 큐브에 장착된 도트매트릭스 화면

CMD(0x54) + IV + r7 + r6 + r5 + r4 + r3 + r2 + r1 + r0 (DotMatrix data of Cube 4) //4번 큐브에 장착된 도트매트릭스 화면

CMD(0x55) + IV + r7 + r6 + r5 + r4 + r3 + r2 + r1 + r0 (DotMatrix data of Cube 5) //5번 큐브에 장착된 도트매트릭스 화면

CMD(0x56) + IV + r7 + r6 + r5 + r4 + r3 + r2 + r1 + r0 (DotMatrix data of Cube 6) //6번 큐브에 장착된 도트매트릭스 화면

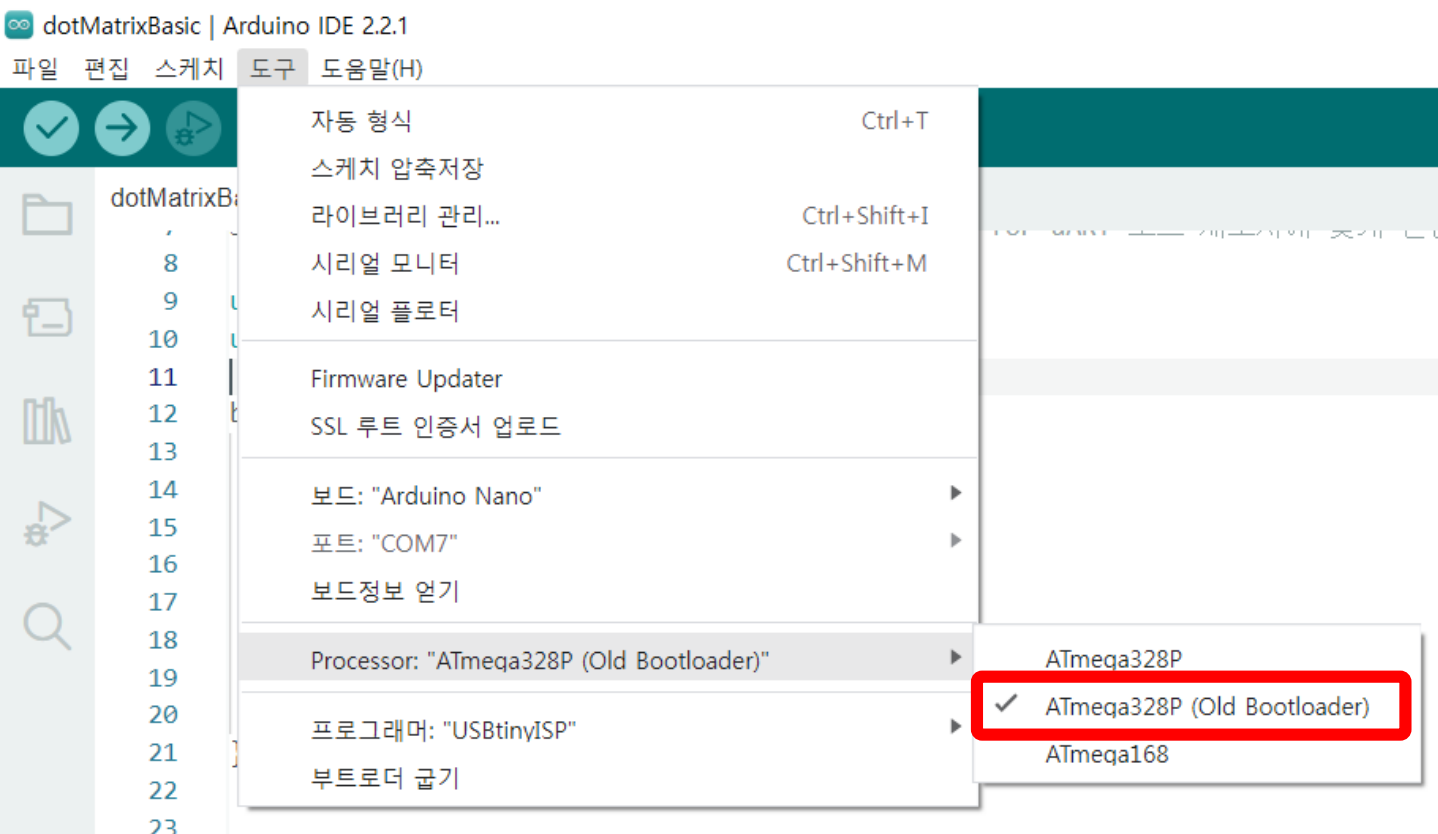
CMD(0x57) + IV + r7 + r6 + r5 + r4 + r3 + r2 + r1 + r0 (DotMatrix data of Cube 7) //7번 큐브에 장착된 도트매트릭스 화면

* 0번 큐브 (STAR)에는 도트매트릭스를 연결할 수 없음

Setting – Arduino Nano



Arduino NANO



아두이노 나노 보드는 종류에 따라 Atmega328P (Old bootloader)로
설정하여야 업로드가 가능한 경우가 있습니다.

Example – controlMotorAngle

설명 : 2개 G큐브(0~1번큐브)의 모터 각도를 제어한다.

STEP 1 : 아두이노 IDE -> 예제 -> Cubeino -> controlMotorAngle 을 오픈한다

STEP 2 : 큐브이노(나노)에 업로드 한다.

STEP 3 : 이 프로젝트에서는 2개 큐브를 사용하므로 G큐브 2개를 준비한다.

STEP 4 : 큐브이노(나노)를 한 개의 G큐브의 확장포트에 꽂아 연결한다 (본 문서 전반부 참조)

STEP 5 : G큐브의 노란색 버튼을 한번 눌러 전원을 켜면 G큐브와 보드가 연결된다.

STEP 6 : 나머지 G큐브의 노란색 버튼을 2번 눌러주면 보드에 연결된 0번 G큐브에 블루투스로 연결된다.

STEP 7 : 모두 연결이 완료되면 아두이노가 연결된 컴퓨터 키보드로 0, 1, 2, 3, 4, 5를 각각 눌러 각 큐브 모터를 확인한다.

```
#include <Servo.h>
#include "Cubeino.h"
#include <SoftwareSerial.h>

#define GCUBE_TOTAL 2 // 이 프로젝트에 사용할 전체 큐브의 숫자
SoftwareSerial myCubeSerial (2, 3); // 2:RX, 3:TX for uART 보드 제조사에 맞게 핀번호 변경하여 사용

unsigned int l_count=0;

void setup()
{
  Serial.begin(115200); // PC에 연결하여 모니터링에 사용하는 UART 통신
  myCubeSerial.begin(115200); // G큐브에 연결하여 사용하는 UART 통신

  Serial.println("Start");

  wait_for_first_cube_connected(myCubeSerial); // G큐브가 켜질 때 까지 기다리기
  wait_for_all_cube_connected(GCUBE_TOTAL, myCubeSerial); // 이 프로젝트에 필요한 큐브들이 모두 연결될 때까지 기다리기

  Serial.println("All cubes are connected");
}

void loop()
{
  if(Serial.available()){
    char c = Serial.read();

    if(c == '0'){
      send_a_cube_angle(0, 300, myCubeSerial);
    }else if(c == '1'){
      send_a_cube_angle(1, 300, myCubeSerial);
    }else if(c == '2'){
      send_a_cube_angle(0, -300, myCubeSerial);
    }else if(c == '3'){
      send_a_cube_angle(1, -300, myCubeSerial);
    }else if(c == '4'){
      stop_all_cube_motor(GCUBE_TOTAL, myCubeSerial);
    }else if(c == '5'){
      send_multi_cube_angle(GCUBE_TOTAL, 500,500,0,0, myCubeSerial);
    }
  }
}
```

Example – controlMotorSpeed

설명 : 2개 G큐브(0~1번큐브)의 모터 속도를 제어한다.

STEP 1 : 아두이노 IDE -> 예제 -> Cubeino -> controlMotorSpeed 을 오픈한다

STEP 2 : 큐브이노(나노)에 업로드 한다.

STEP 3 : 이 프로젝트에서는 2개 큐브를 사용하므로 G큐브 2개를 준비한다.

STEP 4 : 큐브이노(나노)를 한 개의 G큐브의 확장포트에 꽂아 연결한다 (본 문서 전반부 참조)

STEP 5 : G큐브의 노란색 버튼을 한번 눌러 전원을 켜면 G큐브와 보드가 연결된다.

STEP 6 : 나머지 G큐브의 노란색 버튼을 2번 눌러주면 보드에 연결된 0번 G큐브에 블루투스로 연결된다.

STEP 7 : 모두 연결이 완료되면 아두이노가 연결된 컴퓨터 키보드로 0, 1, 2, 3, 4, 5를 각각 눌러 각 큐브 모터를 확인한다.

```
#include <Servo.h>
#include "Cubeino.h"
#include <SoftwareSerial.h>
```

```
#define GCUBE_TOTAL 2 // 이 프로젝트에 사용할 전체 큐브의 숫자
SoftwareSerial myCubeSerial (2, 3); // 2:RX, 3:TX for uART 보드 제조사에 맞게 핀번호 변경하여 사용
```

```
unsigned int l_count=0;
```

```
void setup()
```

```
{
  Serial.begin(115200); // PC에 연결하여 모니터링에 사용하는 UART 통신
  myCubeSerial.begin(115200); // G큐브에 연결하여 사용하는 UART 통신
```

```
  Serial.println("Start");
```

```
  wait_for_first_cube_connected(myCubeSerial); // G큐브가 켜질 때 까지 기다리기
  wait_for_all_cube_connected(GCUBE_TOTAL, myCubeSerial); // 이 프로젝트에 필요한 큐브들이 모두 연결될 때까지
  기다리기
```

```
  Serial.println("All cubes are connected");
```

```
}
```

```
void loop()
```

```
{
  if(Serial.available()){
    char c = Serial.read();
```

```
    if(c == '0'){
      send_a_cube_speed(0, 100, myCubeSerial);
```

```
    }else if(c == '1'){
      send_a_cube_speed(1, 100, myCubeSerial);
```

```
    }else if(c == '2'){
      send_a_cube_speed(0, 50, myCubeSerial);
```

```
    }else if(c == '3'){
      send_a_cube_speed(1, 50, myCubeSerial);
```

```
    }else if(c == '4'){
      stop_all_cube_motor(GCUBE_TOTAL, myCubeSerial);
```

```
    }else if(c == '5'){
      send_multi_cube_speed(GCUBE_TOTAL, 100,100,0,0,0,0,0, myCubeSerial); //G0:100, G1:100, G2.....:0
```

```
    }
```

```
  }
```

```
}
```


Example – controlPingPong

설명 : 근접센서 측정으로 핑퐁 오토카를 제어한다

STEP 1 : 아두이노 IDE -> 예제 -> Cubeino -> controlPingPong 을 오픈한다

STEP 2 : 큐브이노(나노)에 업로드 한다.

STEP 3 : 이 프로젝트에서는 핑퐁오토카를 사용하므로 2개 큐브로 오토카를 조립하여 준비한다.

STEP 4 : 큐브이노(나노)를 한 개의 G큐브의 확장포트에 꽂아서 연결한다 (본 문서 전반부 참조)

STEP 5 : G큐브의 노란색 버튼을 한번 눌러 전원을 켜면 G큐브와 보드가 연결된다.

STEP 6 : 나머지 G큐브의 노란색 버튼을 2번 눌러주면 보드에 연결된 0번 G큐브에 블루투스로 연결된다.

STEP 7 : 모두 연결이 완료되면 0번큐브의 근접센서에 손을 갖다 대면 오토카가 움직인다.

```
#include <Servo.h>
#include "Cubeino.h"
#include <SoftwareSerial.h>
```

```
#define GCUBE_TOTAL 2 // 이 프로젝트에 사용할 전체 큐브의 숫자
// #define GCUBE_TOTAL 3 // 이 프로젝트에 사용할 전체 큐브의 숫자
```

```
SoftwareSerial myCubeSerial (2, 3); // 2:RX, 3:TX for uART 보드 제조사에 맞게 핀번호 변경하여 사용
```

```
unsigned int l_count=0;
bool first_trigger = false;
bool second_trigger = false;
```

```
void setup()
```

```
{
  Serial.begin(115200); // PC에 연결하여 모니터링에 사용하는 UART 통신
  myCubeSerial.begin(115200); // G큐브에 연결하여 사용하는 UART 통신

  Serial.println("Start");
```

```
  wait_for_first_cube_connected(myCubeSerial); // G큐브가 켜질 때 까지 기다리기
  wait_for_all_cube_connected(GCUBE_TOTAL, myCubeSerial); // 이 프로젝트에 필요한 큐브들이 모두 연결될 때까지 기다리기
```

```
  Serial.println("All cubes are connected");
  delay(2000); //Wait for starting motion
}
```

```
void loop()
```

```
{
  byte first_data[3], second_data[3];
  get_cube_proxi_value(first_data, 0, myCubeSerial); //0번 큐브의 근접센서 값을 가져온다.
```

```
  if(first_data[2]>180){
    first_trigger = true;
  }else{
    first_trigger = false;
  }
}
```

```
  if(first_trigger){
    first_trigger=false;
    move_pingpong_robot(PINGPONG_AUTOCAR, 10, myCubeSerial); // 10 cm
    rotate_pingpong_robot(PINGPONG_AUTOCAR, 90, myCubeSerial); // 90 degree (+:right turn)
    move_pingpong_robot(PINGPONG_AUTOCAR, 10, myCubeSerial); // 10 cm
    rotate_pingpong_robot(PINGPONG_AUTOCAR, 90, myCubeSerial); // 90 degree (+:right turn)
    move_pingpong_robot(PINGPONG_AUTOCAR, 10, myCubeSerial); // 10 cm
    rotate_pingpong_robot(PINGPONG_AUTOCAR, 90, myCubeSerial); // 90 degree (+:right turn)
  }
}
```

Example – controlServoMotor

설명 : 2개 G큐브(0~1번큐브)에 붙어 있는 서보모터 각도 제어(단 0번은 큐브에 붙어 있는 아두이노/큐브이노에서 제어)

STEP 1 : 아두이노 IDE -> 예제 -> Cubeino -> controlServoMotor 을 오픈한다

STEP 2 : 큐브이노(나노)에 업로드 한다.

STEP 3 : 이 프로젝트에서는 2개 큐브를 사용하므로 G큐브 2개를 준비한다.

STEP 4 : 큐브이노(나노)를 한 개의 G큐브에 점퍼케이블로 시리얼 통신이 되도록 연결한다 (본 문서 전반부 참조)

STEP 5 : G큐브의 노란색 버튼을 한번 눌러 전원을 켜면 G큐브와 보드가 연결된다.

STEP 6 : 나머지 G큐브의 노란색 버튼을 2번 눌러주면 보드에 연결된 0번 G큐브에 블루투스로 연결된다.

STEP 7 : 보드와 연결된 G큐브에서는 아두이노/큐브이노의 4번핀을 서보모터신호 나머지 5V/GND를 서보모터 전원핀에 각각 연결한다. 나머지 G큐브의 4핀 단자에도 서보모터를 연결한다.

STEP 8 : 모두 연결이 완료되면 아두이노가 연결된 컴퓨터 키보드로 0, 1, 2, 3, 4를 각각 눌러 각 서보 모터를 확인한다.

```
#include <Servo.h>
#include "Cubeino.h"
#include <SoftwareSerial.h>
```

```
#define GCUBE_TOTAL 2 // 이 프로젝트에 사용할 전체 큐브의 숫자
#define SERVO_PIN 4 // 서보모터를 연결한 포트
SoftwareSerial myCubeSerial (2, 3); // 2:RX, 3:TX for uART 보드 제조사에 맞게 핀번호 변경하여 사용
Servo myServo;
```

```
byte servo_angle =0;
```

```
void setup()
```

```
{
  Serial.begin(115200); // PC에 연결하여 모니터링에 사용하는 UART 통신
  myCubeSerial.begin(115200); // G큐브에 연결하여 사용하는 UART 통신
  myServo.attach(SERVO_PIN);
```

```
  Serial.println("Start");
```

```
  wait_for_first_cube_connected(myCubeSerial); // G큐브가 켜질 때 까지 기다리기
  wait_for_all_cube_connected(GCUBE_TOTAL, myCubeSerial); // 이 프로젝트에 필요한 큐브들이 모두 연결될 때까지 기다리기
```

```
  Serial.println("All cubes are connected");
```

```
}
```

```
void loop()
```

```
{
```

```
  if(Serial.available()){
    char c = Serial.read();
```

```
    if(c == '0'){
```

```
      send_a_servo_angle(0, servo_angle, myServo, myCubeSerial);
```

```
    }else if(c == '1'){
```

```
      send_a_servo_angle(1, servo_angle, myServo, myCubeSerial);
```

```
    }else if(c == '2'){
```

```
      send_multi_servo_angle(GCUBE_TOTAL, servo_angle,servo_angle, 0,0,0,0,0, myServo, myCubeSerial);
```

```
    }else if(c == '3'){
```

```
      send_multi_servo_angle(GCUBE_TOTAL, 90,90,0,0,0,0,0, myServo, myCubeSerial);
```

```
    }else if(c == '4'){
```

```
    }
```

```
  }
```

```
  servo_angle ++;
```

```
  servo_angle=servo_angle%180;
```

```
}
```

Example – dotMatrixBasic

설명 : 2개 G큐브(1~2번큐브)에 붙어 있는 도트매트릭스에 이미지를 표시한다.

STEP 1 : 아두이노 IDE -> 예제 -> Cubeino -> dotMatrixBasic 를 오픈한다

STEP 2 : 큐브이노(나노)에 업로드 한다.

STEP 3 : 이 프로젝트에서는 3개 큐브를 사용하므로 G큐브 2개를 준비한다.

STEP 4 : 큐브이노(나노)를 한 개의 G큐브의 확장포트에 꽂아 연결한다 (본 문서 전반부 참조)

STEP 5 : 위 G큐브의 노란색 버튼을 한번 눌러 전원을 켜면 G큐브와 보드가 연결된다.

STEP 6 : 나머지 2개의 G큐브에 도트매트릭스를 꽂은 상태에서 노란색 버튼을 2번씩 눌러주면 보드에 연결된 0번 G큐브에 블루투스도 연결된다.

STEP 7 : 모두 연결이 완료되면 마지막 연결된 G큐브의 도트매트릭스에 하트 모양이 흘러간다.

```
#include <Servo.h>
#include "Cubeino.h"
#include "Cubeino.h"
#include <SoftwareSerial.h>
```

```
#define GCUBE_TOTAL 3 // 이 프로젝트에 사용할 전체 큐브의 숫자
SoftwareSerial myCubeSerial (2, 3); // 2:RX, 3:TX for uART 보드 제조사에 맞게 핀번호 변경하여 사용
```

```
unsigned int l_count=0;
unsigned int roll_index=0;
```

```
byte roll_Image[] = {
  B00000000,
  B00000000,
  B00011000,
  B00111100,
  B00111100,
  B00111100,
  B00011000,
  B00000000,
  B00000000
};
```

```
void setup()
{
  Serial.begin(115200); // PC에 연결하여 모니터링에 사용하는 UART 통신
  myCubeSerial.begin(115200); // G큐브에 연결하여 사용하는 UART 통신

  Serial.println("Start");
```

```
  wait_for_first_cube_connected(myCubeSerial); // G큐브가 켜질 때 까지 기다리기
  wait_for_all_cube_connected(GCUBE_TOTAL, myCubeSerial); // 이 프로젝트에 필요한 큐브들이 모두 연결될 때까지 기다리기

  Serial.println("All cubes are connected");
}
```

```
void loop()
{
  send_matrix_still_image(1, full_Image, myCubeSerial);
  delay(50);
  send_matrix_still_image(2, full_Image, myCubeSerial);
  delay(1000);
  send_matrix_still_image(1, null_Image, myCubeSerial);
  send_matrix_still_image(2, null_Image, myCubeSerial);
  delay(500);

  byte duration = 5; // 롤링시간(초) 계속하려면 0
  start_matrix_roll_image(GCUBE_TOTAL, roll_Image, duration, myCubeSerial);
}
```

Example – dotMatrixArt

설명 : 4개 G큐브(1~4번큐브)에 붙어 있는 도트매트릭스에 예술적인 이모티콘을 표시한다.

STEP 1 : 아두이노 IDE -> 예제 -> Cubeino -> dotMatrixArt 를 오픈한다

STEP 2 : 큐브이노(나노)에 업로드 한다.

STEP 3 : 이 프로젝트에서는 5개 큐브를 사용하므로 G큐브 5개를 준비한다.

STEP 4 : 큐브이노(나노)를 한 개의 G큐브의 확장포트에 꽂아 연결한다 (본 문서 전반부 참조)

STEP 5 : 위 G큐브의 노란색 버튼을 한번 눌러 전원을 켜면 G큐브와 보드가 연결된다.

STEP 6 : 나머지 4개의 G큐브에 도트매트릭스를 꽂은 상태에서 노란색 버튼을 2번씩 눌러주면 보드에 연결된 0번 G큐브에 블루투스로 연결된다.

STEP 7 : 모두 연결이 완료되면 마지막 연결된 G큐브의 도트매트릭스에 하트 모양이 흘러간다.

```
#include <Servo.h>
#include "Cubeino.h"
#include "dotMatrix.h"
#include <SoftwareSerial.h>
```

```
#define GCUBE_TOTAL 5 // 이 프로젝트에 사용할 전체 큐브의 숫자
SoftwareSerial myCubeSerial (2, 3); // 2:RX, 3:TX for uART 보드 제조사에 맞게 핀번호 변경하여 사용
```

```
unsigned int l_count=0;
unsigned int roll_index=0;
```

```
byte roll_Image[] = {
  B00000000,
  B00000000,
  B00011000,
  B00111100,
  B00111100,
  B00011000,
  B00000000,
  B00000000
};
```

```
byte set_Image[] = {
  B00000000,
  B00000000,
  B00011000,
  B00111100,
  B00111100,
  B00011000,
  B00000000,
  B00000000
};
```

```
byte inv_Image[] = {
  B11111111,
  B11111111,
  B11100111,
  B11000011,
  B11000011,
  B11100111,
  B11111111,
  B11111111
};
```

```
byte cross_Image[] = {
  B00011000,
  B00011000,
  B00011000,
  B11111111,
  B11111111,
  B00011000,
  B00011000,
  B00011000
};
```

```
void setup()
```

```
{
  Serial.begin(115200); // PC에 연결하여 모니터링에 사용하는 UART 통신
  myCubeSerial.begin(115200); // G큐브에 연결하여 사용하는 UART 통신
```

```
  Serial.println("Start");
```

```
  wait_for_first_cube_connected(myCubeSerial); // G큐브가 켜질 때 까지 기다리기
```

```
  wait_for_all_cube_connected(GCUBE_TOTAL, myCubeSerial); // 이 프로젝트에
  필요한 큐브들이 모두 연결될 때까지 기다리기
```

```
  Serial.println("All cubes are connected");
  randomSeed(analogRead(0));
```

```
}
```

```
void loop()
```

```
{
  byte rand_img=random(3); // 0, 1, 2
  byte rand_cube=random(1,5); // 1, 2, 3, 4
  if(rand_img==2) send_matrix_still_image(rand_cube, cross_Image,
  myCubeSerial);
  else if(rand_img==1) send_matrix_still_image(rand_cube, set_Image,
  myCubeSerial);
  else send_matrix_still_image(rand_cube, inv_Image, myCubeSerial);
}
```

Example – dotMatrixRolling

설명 : 0번 G큐브에서 가속도값을 가져와 나머지 G큐브(1번~)를 제어하는 예시

STEP 1 : 아두이노 IDE -> 예제 -> Cubeino -> dotMatrixRolling 를 오픈한다

STEP 2 : 큐브이노(나노)에 업로드 한다.

STEP 3 : 이 프로젝트에서는 2개 큐브를 사용하므로 G큐브 2개를 준비한다.

STEP 4 : 큐브이노(나노)를 한 개의 G큐브의 확장포트에 꽂아 연결한다 (본 문서 전반부 참조)

STEP 5 : G큐브의 노란색 버튼을 한번 눌러 전원을 켜면 G큐브와 보드가 연결된다.

STEP 6 : 나머지 G큐브에 도트매트릭스를 꽂은 뒤 노란색 버튼을 2번 눌러주면 보드에 연결된 0번 G큐브에 블루투스로 연결된다.

STEP 7 : 모두 연결이 완료되면 보드와 연결된 G큐브를 손으로 움직여 각도 값에 따라 1번 G큐브의 도트매트릭스의 이미지가 동기되어 움직인다.

```
#include <Servo.h>
```

```
#include "Cubeino.h"
```

```
#include <SoftwareSerial.h>
```

```
#define GCUBE_TOTAL 2 // 이 프로젝트에 사용할 전체 큐브의 숫자
```

```
SoftwareSerial myCubeSerial (2, 3); // 2:RX, 3:TX for uART 보드 제조사에 맞게 핀번호 변경하여 사용
```

```
unsigned int l_count=0;
```

```
unsigned int roll_index=0;
```

```
byte roll_Image[] = {
```

```
  B00000000,
```

```
  B00000000,
```

```
  B00011000,
```

```
  B00111100,
```

```
  B00111100,
```

```
  B00011000,
```

```
  B00000000,
```

```
  B00000000
```

```
};
```

```
void setup()
```

```
{
```

```
  Serial.begin(115200); // PC에 연결하여 모니터링에 사용하는 UART 통신
```

```
  myCubeSerial.begin(115200); // G큐브에 연결하여 사용하는 UART 통신
```

```
  Serial.println("Start");
```

```
  wait_for_first_cube_connected(myCubeSerial); // G큐브가 켜질 때 까지 기다리기
```

```
  wait_for_all_cube_connected(GCUBE_TOTAL, myCubeSerial); // 이 프로젝트에 필요한 큐브들이 모두 연결될 때까지 기다리기
```

```
  Serial.println("All cubes are connected");
```

```
}
```

```
void loop()
```

```
{
```

```
  int8_t temp[3];
```

```
  get_cube_acc_value(temp, 0, myCubeSerial); // Get x, y accelerometer value of the GCube 0
```

```
  Serial.print("x: ");
```

```
  Serial.print((int8_t)temp[1]);
```

```
  Serial.print(" y: ");
```

```
  Serial.print((int8_t)temp[2]);
```

```
  Serial.println("");
```

```
  // Shift image along the acc data with Sync. all connected GCubes
```

```
  start_matrix_shift_image(0, GCUBE_TOTAL, roll_Image, temp[1], temp[2], myCubeSerial);
```

```
}
```

Example – lineTrackingHusky

설명 : 핑퐁로봇 오토카를 허스키렌즈-큐브이노로 라인을 따라가도록 한다

STEP 1 : 아두이노 IDE -> 예제 -> Cubeino -> lineTrackingHusky 을 오픈한다

STEP 2 : 큐브이노(나노)에 업로드 한다.

STEP 3 : 이 프로젝트에서는 핑퐁오토카를 사용하므로 2개 큐브로 오토카를 조립하여 준비한다.

STEP 4 : 큐브이노(나노)를 한 개의 G큐브의 확장포트에 꽂아서 연결한다 (본 문서 전반부 참조)

STEP 5 : 허스키렌즈를 큐브이노 I2C에 연결하고 허스키렌즈 지지대를 이용하여 오토카몸체에 조립한다.

STEP 5 : G큐브의 노란색 버튼을 한번 눌러 전원을 켜면 G큐브와 보드가 연결된다.

STEP 6 : 나머지 G큐브의 노란색 버튼을 2번 눌러주면 보드에 연결된 0번 G큐브에 블루투스로 연결된다.

STEP 7 : 모두 연결이 완료되면 바닥에 붙여진(그려진) 라인을 따라 움직인다.

```
#include "HUSKYLENS.h"
#include "SoftwareSerial.h"
#include "Servo.h"
#include "Cubeino.h"

#define GCUBE_TOTAL 2 // 이 프로젝트에 사용할 전체 큐브의 숫자

SoftwareSerial myCubeSerial (2, 3); // 2:RX, 3:TX for uART 보드 제조사에 맞게 핀번호 변경하여 사용

HUSKYLENS huskylens;
//HUSKYLENS green line >> SDA; blue line >> SCL
int ID1 = 1;

void setup() {
    Serial.begin(115200);
    myCubeSerial.begin(115200); // G큐브에 연결하여 사용하는 UART 통신

    Serial.println("Start Huskylens Line Tracking");

    wait_for_first_cube_connected(myCubeSerial); // G큐브가 켜질 때 까지 기다리기
    wait_for_all_cube_connected(GCUBE_TOTAL, myCubeSerial); // 이 프로젝트에 필요한 큐브들이 모두 연결될 때까지 기다리기

    Wire.begin();
    while (!huskylens.begin(Wire))
    {
        Serial.println(F("Begin failed!"));
        Serial.println(F("1.Please recheck the W\"Protocol TypeW\" in HUSKYLENS (General Settings>>Protol Type>>I2C)"));
        Serial.println(F("2.Please recheck the connection."));
        delay(100);
    }
    huskylens.writeAlgorithm(ALGORITHM_LINE_TRACKING); //Switch the algorithm to line tracking.
}

int left = 0, right = 0;

void loop() {
    int32_t error;
    if (!huskylens.request(ID1)) {Serial.println(F("Fail to request data from HUSKYLENS, recheck the connection!"));left = 0; right = 0;}
    else if(!huskylens.isLearned()) {Serial.println(F("Nothing learned, press learn button on HUSKYLENS to learn one!"));left = 0; right = 0;}
    else if(!huskylens.available()) Serial.println(F("No block or arrow appears on the screen!"));
    else{
        HUSKYLENSResult result = huskylens.read();
        // Calculate the error:
        error = (int32_t)result.xTarget - (int32_t)160;
        // left = 40 - error * 0.06;
        // right = 40 + error * 0.06;
        left = 30 - error * 0.08;
        right = 30 + error * 0.08;
        // left = 20 - error * 0.1;
        // right = 20 + error * 0.1;
        if(left<0) left=10; else if(left>100) left=100;
        if(right<0) right=10; else if(right>100) right=100;
    }

    Serial.println(String()+left+", "+right);
    send_multi_cube_speed(GCUBE_TOTAL, -1*right,left,0,0,0,0,0, myCubeSerial); // G0:-1*right, G1:left, G2.....:0
}
```


Example – lineTrackingIR

설명 : 핑퐁로봇 오토카를 IR센서2개-큐브이노로 라인을 따라가도록 한다

STEP 1 : 아두이노 IDE -> 예제 -> Cubeino -> lineTrackingIR 을 오픈한다

STEP 2 : 큐브이노(나노)에 업로드 한다.

STEP 3 : 이 프로젝트에서는 핑퐁오토카를 사용하므로 2개 큐브로 오토카를 조립하여 준비한다.

STEP 4 : 큐브이노(나노)를 한 개의 G큐브의 확장포트에 꽂아서 연결한다 (본 문서 전반부 참조)

STEP 5 : 적외선센서 2개를 큐브이노 A2(좌), A3(우)에 연결하고 오토카몸체 받침에 집게로 연결한다.

STEP 5 : G큐브의 노란색 버튼을 한번 눌러 전원을 켜면 G큐브와 보드가 연결된다.

STEP 6 : 나머지 G큐브의 노란색 버튼을 2번 눌러주면 보드에 연결된 0번 G큐브에 블루투스로 연결된다.

STEP 7 : 모두 연결이 완료되면 바닥에 붙여진(그려진) 라인을 따라 움직인다.

```
#include <Servo.h>
#include "Cubeino.h"
#include <SoftwareSerial.h>

#define GCUBE_TOTAL 2 // 이 프로젝트에 사용할 전체 큐브의 숫자
SoftwareSerial myCubeSerial (2, 3); // 2:RX, 3:TX for uART 보드 제조사에 맞게 핀번호 변경하여 사용

unsigned int l_count=0;
unsigned int left=0;
unsigned int right=0;

void setup()
{
    Serial.begin(115200); // PC에 연결하여 모니터링에 사용하는 UART 통신
    myCubeSerial.begin(115200); // G큐브에 연결하여 사용하는 UART 통신

    Serial.println("Start");

    wait_for_first_cube_connected(myCubeSerial); // G큐브가 켜질 때 까지 기다리기
    wait_for_all_cube_connected(GCUBE_TOTAL, myCubeSerial); // 이 프로젝트에 필요한 큐브들이 모두 연결될 때까지 기다리기

    Serial.println("All cubes are connected");
}

void loop()
{
    // send_a_Cube_Speed(0, 100, myCubeSerial);

    left= analogRead(A2);
    right= analogRead(A3);

    Serial.print("LEFT: ");
    Serial.print(left);
    Serial.print(" RIGHT: ");
    Serial.println(right);

    // nominal speed 30 with left sensor:right sensor
    if(left < 100 & right < 100) send_multi_cube_speed(GCUBE_TOTAL, -30, 30, 0, 0, 0, 0, 0, 0, myCubeSerial);
    else if(left > 100) send_multi_cube_speed(GCUBE_TOTAL, -22, 38, 0, 0, 0, 0, 0, 0, myCubeSerial);
    else if(right > 100) send_multi_cube_speed(GCUBE_TOTAL, -38, 22, 0, 0, 0, 0, 0, 0, myCubeSerial);
    else send_multi_cube_speed(GCUBE_TOTAL, 0,0,0,0,0,0,0,0,0, myCubeSerial);
}
```

Example – readCubeExternalSensor

설명 : G큐브에 꽂혀 있는여러가지 센서를 읽는 예시

STEP 1 : 아두이노 IDE -> 예제 -> Cubeino -> readCubeExternalSensor 을 오픈한다

STEP 2 : 큐브이노(나노)에 업로드 한다.

STEP 3 : 이 프로젝트에서는 2개 큐브를 사용하므로 G큐브 2개를 준비한다.

STEP 4 : 큐브이노(나노)를 한 개의 G큐브의 확장포트에 꽂아 연결한다 (본 문서 전반부 참조)

STEP 5 : G큐브의 노란색 버튼을 한번 눌러 전원을 켜면 G큐브와 보드가 연결된다.

STEP 6 : 나머지 G큐브의 노란색 버튼을 2번 눌러주면 보드에 연결된 0번 G큐브에 블루투스로 연결된다.

*주의사항 : 나머지 큐브에는 측정하고자 하는 센서를 확장포트에 미리 연결한 상태에서 전원을 켜다.

STEP 7 : 모두 연결이 완료되면 G큐브 0번 1번의 가속도센서, 근접센서, 버튼 눌림 상태값을 보여준다.

```
#include <Servo.h>
#include "Cubeino.h"
#include <SoftwareSerial.h>

#define GCUBE_TOTAL 2 // 이 프로젝트에 사용할 전체 큐브의 숫자
SoftwareSerial myCubeSerial (2, 3); // 2:RX, 3:TX for uART 보드 제조사에 맞게 핀번호 변경하여 사용

unsigned int l_count=0;

void setup()
{
  Serial.begin(115200); // PC에 연결하여 모니터링에 사용하는 UART 통신
  myCubeSerial.begin(115200); // G큐브에 연결하여 사용하는 UART 통신

  Serial.println("Start");

  wait_for_first_cube_connected(myCubeSerial); // G큐브가 켜질 때 까지 기다리기
  wait_for_all_cube_connected(GCUBE_TOTAL, myCubeSerial); // 이 프로젝트에 필요한 큐브들이 모두 연결될 때까지 기다리기

  Serial.println("Sensor Hub start");
}

void loop()
{
  byte sensor_data[3];
  int sensor_result = 0;

  // 함수에서 선택한 큐브에 해당 센서가 미리 꽂혀 있어야 함.
  sensor_result = get_cube_ext_sensor_value(sensor_data, 1, GCUBE_EXT_SENSOR_LIGHT, myCubeSerial); //1번 큐브
  의 외부포트에 꽂혀 있는 (선택된)센서 값을 가져온다.
  Serial.print(" Light sensor value:");
  Serial.print(sensor_result);
  Serial.print(" ### ");

  sensor_result = get_cube_ext_sensor_value(sensor_data, 1, GCUBE_EXT_SENSOR_SOUND, myCubeSerial); //1번 큐브
  의 외부포트에 꽂혀 있는 (선택된)센서 값을 가져온다.
  Serial.print(" Light sensor value:");
  Serial.print(sensor_result);
  Serial.print(" ### ");

  sensor_result = get_cube_ext_sensor_value(sensor_data, 1, GCUBE_EXT_SENSOR_VOLUME, myCubeSerial); //1번 큐브
  의 외부포트에 꽂혀 있는 (선택된)센서 값을 가져온다.
  Serial.print(" Volume sensor value:");
  Serial.print(sensor_result);
  Serial.print(" ### ");
```

```

    sensor_result = get_cube_ext_sensor_value(sensor_data, 1, GCUBE_EXT_SENSOR_MAGNETIC, myCubeSerial); //1 번
    큐브의 외부포트에 꽂혀 있는 (선택된)센서 값을 가져온다.
    Serial.print(" Magnetic sensor value:");
    Serial.print(sensor_result);
    Serial.print(" ### ");

    sensor_result = get_cube_ext_sensor_value(sensor_data, 1, GCUBE_EXT_SENSOR_ULTRASONIC, myCubeSerial); //1
    번 큐브의 외부포트에 꽂혀 있는 (선택된)센서 값을 가져온다.
    Serial.print(" Ultrasonic sensor value:");
    Serial.print(sensor_result);
    Serial.print(" ### ");

    sensor_result = get_cube_ext_sensor_value(sensor_data, 1, GCUBE_EXT_SENSOR_TEMPRATURE, myCubeSerial); //1
    번 큐브의 외부포트에 꽂혀 있는 (선택된)센서 값을 가져온다.
    Serial.print(" Temperature sensor value:");
    Serial.print(sensor_data[1]);
    Serial.print(".");
    Serial.print(sensor_data[2]);
    Serial.print("degree ");
    Serial.print(" ### ");

    sensor_result = get_cube_ext_sensor_value(sensor_data, 1, GCUBE_EXT_SENSOR_COLOR_KEY, myCubeSerial); //1번
    큐브의 외부포트에 꽂혀 있는 칼라센서 키값을 가져온다.
    Serial.print(" Color sensor KEY value:");
    Serial.print(sensor_data[2]);
    Serial.print(" ### ");

    sensor_result = get_cube_ext_sensor_value(sensor_data, 1, GCUBE_EXT_SENSOR_COLOR_VALUE, myCubeSerial);
    //1 번 큐브의 외부포트에 꽂혀 있는 (선택된)센서 값을 가져온다.
    Serial.print(" Color sensor value:");
    Serial.print(" Red:");
    Serial.print(sensor_data[0]);
    Serial.print(" Green:");
    Serial.print(sensor_data[1]);
    Serial.print(" Blue:");
    Serial.print(sensor_data[2]);
    Serial.print(" ### ");

    sensor_result = get_cube_ext_sensor_value(sensor_data, 1, GCUBE_EXT_SENSOR_ANALOG, myCubeSerial); //1 번 큐
    브의 외부포트에 꽂혀 있는 (선택된)센서 값을 가져온다.
    Serial.print(" 8bit:");
    Serial.print(sensor_data[1]);
    Serial.print(" 12bit:");
    Serial.print(sensor_result);
    Serial.println(" ### ");

    delay(50); // +50msec(@sendData) = 100msec
}

```

Example – readCubeColorSensor

설명 : G큐브에 꽂혀 있는 칼라센서를 읽는 예시

STEP 1 : 아두이노 IDE -> 예제 -> Cubeino -> readCubeColorSensor 을 오픈한다

STEP 2 : 큐브이노(나노)에 업로드 한다.

STEP 3 : 이 프로젝트에서는 3개 큐브를 사용하므로 G큐브 3개를 준비한다.

STEP 4 : 큐브이노(나노)를 한 개의 G큐브의 확장포트에 꽂아 연결한다 (본 문서 전반부 참조)

STEP 5 : G큐브의 노란색 버튼을 한번 눌러 전원을 켜면 G큐브와 보드가 연결된다.

STEP 6 : 나머지 2개 G큐브의 노란색 버튼을 2번 눌러주면 보드에 연결된 0번 G큐브에 블루투스로 연결된다.

*주의사항 : G큐브 1번에는 칼라센서를 연결하고 2번에는 도트매트릭스를 연결한 후 전원을 켜야한다.

STEP 7 : 모두 연결이 완료된 후 일반 색종이 위에 칼라센서를 5mm 내외 높이에 두면 색상을 인식한다.

```
#include <Servo.h>
#include "Cubeino.h"
#include "dotMatrix.h"
#include <SoftwareSerial.h>

#define GCUBE_TOTAL 3 // 이 프로젝트에 사용할 전체 큐브의 숫자
SoftwareSerial myCubeSerial (2, 3); // 2:RX, 3:TX for uART 보드 제조사에 맞게 핀번호 변경하여 사용

unsigned int l_count=0;

void setup()
{
    Serial.begin(115200); // PC에 연결하여 모니터링에 사용하는 UART 통신
    myCubeSerial.begin(115200); // G큐브에 연결하여 사용하는 UART 통신
    Serial.println("Start");
    wait_for_first_cube_connected(myCubeSerial); // G큐브가 켜질 때 까지 기다리기
    wait_for_all_cube_connected(GCUBE_TOTAL, myCubeSerial); // 이 프로젝트에 필요한 큐브들이 모두 연결될 때까지 기다리기

    Serial.println("Sensor Hub start");
}

void loop()
{
    byte sensor_data[3];
    byte analogPeriod = 0;
    byte bitFlag = 0;
    int sensor_result = 0;

    sensor_result = get_cube_ext_sensor_value(sensor_data, 1, GCUBE_EXT_SENSOR_COLOR_KEY, myCubeSerial); //1번 큐브의 외부포트에 꽂혀 있
    는 칼라센서 키값을 가져온다.
    Serial.print(" Color sensor KEY value:");
    Serial.print(sensor_data[2]);
    Serial.print(" ### ");

    switch(sensor_data[2]){
        case 0:    send_matrix_still_image(2, letter_0, myCubeSerial);           break;
        case 1:    send_matrix_still_image(2, letter_1, myCubeSerial);         break;
        case 2:    send_matrix_still_image(2, letter_2, myCubeSerial);         break;
        case 3:    send_matrix_still_image(2, letter_3, myCubeSerial);         break;
        case 4:    send_matrix_still_image(2, letter_4, myCubeSerial);         break;
        case 5:    send_matrix_still_image(2, letter_5, myCubeSerial);         break;
        case 6:    send_matrix_still_image(2, letter_6, myCubeSerial);         break;
        case 7:    send_matrix_still_image(2, letter_7, myCubeSerial);         break;
        default:    send_matrix_still_image(2, full_Image, myCubeSerial);      break;
    }

    sensor_result = get_cube_ext_sensor_value(sensor_data, 1, GCUBE_EXT_SENSOR_COLOR_VALUE, myCubeSerial); //1번 큐브의 외부포트에 꽂혀
    있는 (선택된)센서 값을 가져온다.
    Serial.print(" Color sensor value:");
    Serial.print(" Red:");
    Serial.print(sensor_data[0]);
    Serial.print(" Green:");
    Serial.print(sensor_data[1]);
    Serial.print(" Blue:");
    Serial.print(sensor_data[2]);
    Serial.println(" ### ");

    delay(50); // +50msec(@sendData) = 100msec
}
```

Example – readCubeArduinoSensor

설명 : G큐브에 꽂혀 있는 큐브이노/나노 보드의 아날로그 포트값을 읽거나 디지털 포트를 제어할 때

STEP 1 : 아두이노 IDE -> 예제 -> Cubeino -> readCubeArduinoSensor 을 오픈한다

STEP 2 : 큐브이노(나노)에 업로드 한다.

STEP 3 : 이 프로젝트에서는 2개 큐브를 사용하므로 G큐브 2개를 준비한다.

STEP 4 : 큐브이노(나노)를 한 개의 G큐브의 확장포트에 꽂아 연결한다 (본 문서 전반부 참조)

STEP 5 : G큐브의 노란색 버튼을 한번 눌러 전원을 켜면 G큐브와 보드가 연결된다.

STEP 6 : 나머지 G큐브의 노란색 버튼을 2번 눌러주면 보드에 연결된 0번 G큐브에 블루투스로 연결된다.

*주의사항 : 나머지 큐브에는 아날로그 0번에 임의의 센서를 연결한 큐브이노(나노)를 확장포트에 연결한다.

STEP 7 : 모두 연결이 완료되면 G큐브 0번 1번의 가속도센서, 근접센서, 버튼 눌림 상태값을 보여준다.

```
#include <Servo.h>
#include "Cubeino.h"
#include <SoftwareSerial.h>

#define GCUBE_TOTAL 2 // 이 프로젝트에 사용할 전체 큐브의 숫자
SoftwareSerial myCubeSerial (2, 3); // 2:RX, 3:TX for uART 보드 제조사에 맞게 핀번호 변경하여 사용

unsigned int l_count=0;

void setup()
{
    Serial.begin(115200); // PC에 연결하여 모니터링에 사용하는 UART 통신
    myCubeSerial.begin(115200); // G큐브에 연결하여 사용하는 UART 통신

    Serial.println("Start");

    wait_for_first_cube_connected(myCubeSerial); // G큐브가 켜질 때 까지 기다리기
    wait_for_all_cube_connected(GCUBE_TOTAL, myCubeSerial); // 이 프로젝트에 필요한 큐브들이 모두 연결될 때까지 기다리기

    Serial.println("Arduino sensor reading start");
}

void loop()
{
    int sensor_value;
    uint16_t iflag = 0;

    sensor_value = get_cube_arduino_analog_value(1, 0, myCubeSerial); //1번 큐브에 꽂혀 있는 아두이노의 아날로그
    포트 0번의 값을 읽어 온다.
    Serial.print(" Analog value: ");
    Serial.println(sensor_value);

    // 40*50 = 2000ms (2초) 주기로 1번 큐브에 꽂혀 있는 아두이노의 디지털 포트 전체를 끄고 켜다.
    if (iflag%40==0)
        set_cube_arduino_digital_value(1, 0xFF, 0xFF, myCubeSerial); // 1번 큐브에 꽂혀 있는 아두이노의
    디지털 포트의 값을 모두 1로 세팅한다.
    else if(iflag%40==20)
        set_cube_arduino_digital_value(1, 0x00, 0x00, myCubeSerial); // 1번 큐브에 꽂혀 있는 아두이노의
    디지털 포트의 값을 모두 0로 세팅한다.

    iflag++;

    delay(50); // +50msec(@sendData) = 100msec
}
```

Example – sensorHub Mode

설명 : G큐브-아두이노/큐브이노를 센서 허브 장치로 이용할 때

STEP 1 : 아두이노 IDE -> 예제 -> Cubeino -> motorControl 을 오픈한다

STEP 2 : 큐브이노(나노)에 업로드 한다.

STEP 3 : 이 프로젝트에서는 1개 큐브를 사용하므로 G큐브 1개를 준비한다.

STEP 4 : 큐브이노(나노)를 한 개의 G큐브의 확장포트에 꽂아 연결한다 (본 문서 전반부 참조)

STEP 5 : G큐브의 노란색 버튼을 한번 눌러 전원을 켜면 G큐브와 보드가 연결된다.

STEP 6 : 모두 연결이 완료되면 G큐브와 연결된 스크래치,엔트리,파이썬 등에서 Hub보드를 읽거나 쓸 수 있다.

```
#include <Servo.h>
#include "Cubeino.h"
#include <SoftwareSerial.h>

#define CUBE_TOTAL 2 // 이 프로젝트에 사용할 전체 큐브의 숫자
SoftwareSerial myCubeSerial (2, 3); // 2:RX, 3:TX for uART 보드 제조사에 맞게 핀번호 변경하여 사용

unsigned int l_count=0;

void setup()
{
  Serial.begin(115200); // PC에 연결하여 모니터링에 사용하는 시리얼 통신
  myCubeSerial.begin(115200); // G큐브에 연결하여 사용하는 시리얼 통신

  Serial.println("Sensor Hub mode start");
}

void loop()
{
  byte req_data[3];
  byte analogPeriod = 0;
  byte bitFlag = 0;

  get_req_command(req_data, myCubeSerial);

  switch(req_data[0]){
    case GCube_UART_REQ_ANALOG_READ: //just one time request fot start
      analogPeriod=req_data[1]; // start analog data transfer in period 1:100msed ~ 100:10sec
      bitFlag=req_data[2];
      break;
    case GCube_UART_REQ_DIGITAL_SET:
      set_multi_digital_port(req_data[1], req_data[2]);
      break;
    default:
      break;
  }

  delay(50); // +50msec(@sendData) = 100msec

  if(analogPeriod>0){
    if(l_count%analogPeriod==0) send_analog_data(bitFlag, myCubeSerial);
  }

  l_count++;
  l_count=l_count%10000;
}
```

Library – Cubeino.h (1)

아두이노-G큐브 연결을 위한 라이브러리는 Cubino.zip 을 아두이노 IDE → 스케치 → 라이브러리 포함하기 --> ZIP 라이브러리 추가에서 추가를 합니다.

아두이노 IDE → 파일 → 예제 → Cubino → 에서 모터 제어와 도트매트릭스 예제를 열어 동작을 확인합니다.

아래는 아두이노 라이브러리 헤더 파일 입니다.

```
#define GCUBE_GET_BOARD_ID 0x10 // 보드의 ID 값 확인 요청
#define GCUBE_CUBEINO_BOARD_ID 0x70 // 제조사 및 모델마다 고유 아이디 부여함, Cubeino : 0x70
#define GCUBE_HUB_BOARD_ID 0x71 // 큐브에 꽂힌 아두이노(큐브이노)가 Sensor Hub 역할을 하는 경우 보드 ID : 0x71
#define GCUBE_GET_CUBE_NUMBER 0x21 // 연결된 큐브 개수 요청
#define GCUBE_SET_ALL_CUBES_CONNECTED 0x2A // 모든 큐브 연결됨

#define GCUBE_REQ_ANALOG_READ 0x11
#define GCUBE_REQ_DIGITAL_SET 0x1A // 아두이노의 디지털 포트 명령 수신

//GCube motor & servo & dot-matrix control
#define GCUBE_SET_A_MOTOR_SPEED 0x30
#define GCUBE_SET_MULTI_MOTOR_SPEED 0x31
#define GCUBE_SET_A_MOTOR_ANGLE 0x38
#define GCUBE_SET_MULTI_MOTOR_ANGLE 0x39
#define GCUBE_SET_A_SERVO_ANGLE 0x40
#define GCUBE_SET_MULTI_SERVO_ANGLE 0x41
#define GCUBE_SET_DOTMATRIX 0x51

//Get GCube sensors
#define GCUBE_GET_CUBE_GYRO_BASE 0xA0 // 큐브의 기울기값(자이로센서) 요청
#define GCUBE_GET_CUBE_PROXI_BASE 0xAA // 큐브의 근접센서 요청
#define GCUBE_GET_CUBE_BUTTON_BASE 0xAB // 큐브의 버튼 상태 요청
#define GCUBE_GET_CUBE_EXT_ANALOG_2BYTE 0xB0 // 큐브의 확장포트 2바이트형 센서 값
#define GCUBE_GET_CUBE_ARDUINO_ANALOG_BASE 0xD0 // 큐브-아두이노의 아날로그값 요청
#define GCUBE_SET_CUBE_ARDUINO_ANALOG_START 0xD8 // 특정 큐브-아두이노의 아날로그 신호 보고 명령 전송
#define GCUBE_SET_CUBE_ARDUINO_DIGITAL_CONTROL 0xD9 // 특정 큐브-아두이노의 디지털 포트 제어 명령 전송
#define GCUBE_GET_CUBE_EXT_COLOR_KEY 0xE0 // 큐브의 확장포트 칼라센서 칼라키 값
#define GCUBE_GET_CUBE_EXT_COLOR_VALUE 0xE8 // 큐브의 확장포트 칼라센서 RGB 값
#define GCUBE_GET_CUBE_ECHO_BASE 0xFF // 단순 Echo 메시지 받는 기능

//GCube external sensor name
#define GCUBE_EXT_SENSOR_LIGHT 0xC1
#define GCUBE_EXT_SENSOR_SOUND 0xC2
#define GCUBE_EXT_SENSOR_VOLUME 0xC3
#define GCUBE_EXT_SENSOR_MAGNETIC 0xC4
#define GCUBE_EXT_SENSOR_ULTRASONIC 0xC5
#define GCUBE_EXT_SENSOR_TEMPRATURE 0xC6
#define GCUBE_EXT_SENSOR_COLOR_KEY 0xC7
#define GCUBE_EXT_SENSOR_COLOR_VALUE 0xC8
#define GCUBE_EXT_SENSOR_ANALOG 0xF7

//Control PingPong robot model
#define PINGPONG_ANTBOT 2 // Gear가 장착된 자동차형 핑퐁로봇 모델
#define PINGPONG_BATTLEBOT 2 // Gear가 장착된 자동차형 핑퐁로봇 모델
#define PINGPONG_AUTOCAR 1 // Gear가 장착된 자동차형 핑퐁로봇 모델
#define PINGPONG_DRAWINGBOT 0 // Gear가 장착되지 않은 자동차형 핑퐁로봇 모델
```


Library – Cubeino.h (2)

//Common util

```
void send_gcube(byte x0, byte x1, byte x2, byte x3, byte x4, byte x5, byte x6, byte x7, byte x8, byte x9, Stream &cubeSerial);
```

```
byte get_iv(byte a);
```

```
void get_cube_echo_value(uint8_t mdata[], Stream &cubeSerial);
```

//Get GCube sensor value

```
void get_cube_acc_value(int8_t mdata[], byte c_number, Stream &cubeSerial);
```

```
void get_cube_proxi_value(uint8_t mdata[], byte c_number, Stream &cubeSerial);
```

```
void get_cube_button_value(uint8_t mdata[], byte c_number, Stream &cubeSerial);
```

```
uint16_t get_cube_ext_sensor_value(uint8_t mdata[], byte c_number, byte s_name, Stream &cubeSerial);
```

//Get analog value or set digital port of GCube-attached Arduino

```
void set_cube_arduino_analog_start(byte c_number, Stream &cubeSerial);
```

```
uint16_t get_cube_arduino_analog_value(byte c_number, byte a_port, Stream &cubeSerial);
```

```
void set_cube_arduino_digital_value(byte c_number, byte port_hi, byte port_lo, Stream &cubeSerial);
```

//Interface between Arduino and the connected GCube

```
void wait_for_first_cube_connected(Stream &cubeSerial);
```

```
void wait_for_hub_cube_connected(Stream &cubeSerial);
```

```
void wait_for_all_cube_connected(byte GCUBE_TOTAL, Stream &cubeSerial);
```

```
void get_req_command(uint8_t mdata[], Stream &cubeSerial);
```

```
void report_analog_data(Stream &cubeSerial);
```

```
void set_digital_port(byte vh, byte vl);
```

//Control GCube's motor and attached servo-motor or matrix

```
void send_a_cube_speed(byte cube_number, char cube_speed, Stream &cubeSerial);
```

```
void send_multi_cube_speed(byte GCUBE_TOTAL, char s0, char s1, char s2, char s3, char s4, char s5, char s6, char s7, Stream &cubeSerial);
```

```
void stop_all_cube_motor(byte GCUBE_TOTAL, Stream &cubeSerial);
```

```
void send_a_cube_angle(byte cube_number, int cube_angle, Stream &cubeSerial);
```

```
void send_multi_cube_angle(byte cube_range, int s0, int s1, int s2, int s3, Stream &cubeSerial);
```

```
void send_a_servo_angle(byte cube_number, byte cube_angle, Servo &svo, Stream &cubeSerial);
```

```
void send_multi_servo_angle(byte GCUBE_TOTAL, byte s0, byte s1, byte s2, byte s3, byte s4, byte s5, byte s6, byte s7, Servo &svo, Stream &cubeSerial);
```

```
void send_matrix_still_image(byte cube_index, byte m[], Stream &cubeSerial);
```

```
void start_matrix_roll_image(byte GCUBE_TOTAL, byte m[], int duration, Stream &cubeSerial);
```

```
void start_matrix_shift_image(byte cubeIndex, byte GCUBE_TOTAL, byte rollImage[], int8_t xp, int8_t yp, Stream &cubeSerial);
```

//Control PingPong robot model with Arduino

```
void move_pingpong_robot(byte gear_flag, int distance, Stream &cubeSerial);
```

```
void rotate_pingpong_robot(byte gear_flag, int angle, Stream &cubeSerial);
```

```
void pen_up(Stream &cubeSerial);
```

```
void pen_down(Stream &cubeSerial);
```

```
void lever_up(Stream &cubeSerial);
```

```
void lever_down(Stream &cubeSerial);
```

```
void gripper_open(Stream &cubeSerial);
```

```
void gripper_close(Stream &cubeSerial);
```

Library – Arduino - API (1)

send_gcube

설명: 큐브로 데이터를 전송하는 함수입니다. 지정된 바이트 값을 큐브로 전송합니다.

인자:
x0 ~ x9: 바이트 값들
cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

wait_for_first_cube_connected

설명: 처음 연결된 큐브를 기다리는 함수입니다. 연결된 큐브에게 보드 ID를 요청하고 응답을 확인하여 처리합니다.

인자: cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

wait_for_hub_cube_connected

설명: 허브 큐브를 기다리는 함수로, 첫 번째 큐브가 연결된 후에 호출되어 허브 큐브로 설정합니다.

인자: cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

wait_for_all_cube_connected

설명: 모든 큐브가 연결될 때까지 기다리는 함수로, 각 큐브에게 연결 여부를 확인합니다.

인자:
GCUBE_TOTAL: 사용할 큐브의 총 수
cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

get_iv

설명: 인텔리센스 값 (Intellicube value)을 계산하는 함수로, 주어진 바이트 값을 변환하여 반환합니다.

인자: a: 변환할 바이트 값
반환 값: 변환된 값 (바이트)

start_matrix_shift_image

설명: 행렬 이미지를 이동시키는 함수로, 큐브에게 행렬 이미지를 전송하여 표시합니다.

인자:
cubeIndex: 큐브의 인덱스 (0은 모든 연결된 큐브에 대한 작업)
GCUBE_TOTAL: 사용할 큐브의 총 수
rollImage[]: 이동할 이미지의 배열
xp: X 축 이동 값
yp: Y 축 이동 값
cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

get_cube_echo_value

설명: 큐브의 음향 반사 값을 가져오는 함수로, 큐브에게 해당 정보를 요청하고 응답을 받습니다.

인자:
mdata[]: 큐브의 음향 반사 값을 저장할 배열
cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

get_cube_acc_value

설명: 큐브의 가속도계 값을 가져오는 함수로, 큐브에게 해당 정보를 요청하고 응답을 받습니다.

인자:
mdata[]: 큐브의 가속도계 값을 저장할 배열
c_number: 큐브의 번호
cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

get_cube_proxi_value

설명: 큐브의 근접 센서 값을 가져오는 함수로, 큐브에게 해당 정보를 요청하고 응답을 받습니다.

인자:
mdata[]: 큐브의 근접 센서 값을 저장할 배열
c_number: 큐브의 번호
cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

get_cube_button_value

설명: 큐브의 버튼 상태 값을 가져오는 함수로, 큐브에게 해당 정보를 요청하고 응답을 받습니다.

인자:
mdata[]: 큐브의 버튼 상태 값을 저장할 배열
c_number: 큐브의 번호
cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

get_cube_ext_sensor_value

설명: 큐브의 외부 센서 값(빛, 소리, 온도 등)을 가져오는 함수로, 큐브에게 해당 정보를 요청하고 응답을 받습니다.

인자:
mdata[]: 큐브의 외부 센서 값을 저장할 배열
c_number: 큐브의 번호
s_name: 외부 센서 종류 (예: GCUBE_EXT_SENSOR_LIGHT)
cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)
반환 값: 외부 센서에 따라 다름 (빛, 소리, 온도 등)

Library – Arduino - API (2)

get_cube_arduino_analog_value

설명: 큐브의 Arduino 아날로그 핀 값(예: 광센서)을 가져오는 함수로, 큐브에게 해당 정보를 요청하고 응답을 받습니다.
인자:

c_number: 큐브의 번호
a_port: Arduino의 아날로그 핀 번호
cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

set_cube_arduino_analog_start

설명: 큐브의 Arduino 아날로그 센서 읽기를 시작하는 함수로, 큐브에게 해당 정보를 요청합니다.
인자:

c_number: 큐브의 번호
cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

get_req_command

설명: 큐브로부터 요청 명령을 가져오는 함수로, 큐브에게 해당 정보를 요청하고 응답을 받습니다.
인자:

mdata[]: 명령 값을 저장할 배열
cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

report_analog_data

설명: 아날로그 데이터를 보고하는 함수로, 아날로그 핀의 값을 읽어 큐브에게 보고합니다.
인자: cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

set_cube_arduino_digital_value

설명: 큐브의 Arduino 디지털 핀에 값을 설정하는 함수로, 큐브에게 해당 정보를 전송합니다.
인자:

c_number: 큐브의 번호
d_port: Arduino의 디지털 핀 번호
value: 설정할 값 (HIGH 또는 LOW)
cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

set_digital_port

설명: D4에서 D13까지의 디지털 포트에 대한 출력 상태를 설정합니다.
인자:

vh: D12에서 D13까지의 포트를 나타내는 4비트 값
vl: D4에서 D11까지의 포트를 나타내는 8비트 값

send_a_cube_speed

설명: 단일 큐브의 모터 속도를 설정합니다.
인자:

cube_number: 큐브의 번호
cube_speed: 설정할 속도 값
cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

send_multi_cube_speed

설명: 다중 큐브의 모터 속도를 설정합니다.
인자:

GCUBE_TOTAL: 사용할 큐브의 총 수
s0 ~ s7: 큐브별로 설정할 속도 값
cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

stop_all_cube_motor

설명: 모든 큐브의 모터를 정지시킵니다.
인자:

GCUBE_TOTAL: 사용할 큐브의 총 수
cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

send_a_cube_angle

설명: 단일 큐브의 모터 각도를 설정합니다.
인자:

cube_number: 큐브의 번호
cube_angle: 설정할 각도 값 (0~180)
cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

send_multi_cube_angle

설명: 다중 큐브의 모터 각도를 설정합니다.
인자:

cube_range: 설정할 큐브의 총 수
s0 ~ s3: 큐브별로 설정할 각도 값 (0~180)
cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

Library – Arduino - API (3)

send_a_servo_angle

설명: 단일 큐브의 서보 각도를 설정합니다.

인자:

- cube_number: 큐브의 번호
- cube_angle: 설정할 각도 값 (0~180)
- svo: Servo 객체
- cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

send_multi_servo_angle

설명: 다중 큐브의 서보 각도를 설정합니다.

인자:

- GCUBE_TOTAL: 사용할 큐브의 총 수
- s0 ~ s7: 큐브별로 설정할 각도 값 (0~180)
- svo: Servo 객체
- cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

send_matrix_still_image

설명: 단일 큐브의 도트 매트릭스에 정지 이미지를 설정합니다.

인자:

- cube_index: 큐브의 인덱스
- m[]: 8개의 값으로 이루어진 도트 매트릭스 이미지
- cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

start_matrix_roll_image

설명: 여러 큐브에서 도트 매트릭스 이미지를 롤링하는 함수입니다. 이미지를 롤링하면서 각 큐브에 대해 매트릭스를 업데이트하고 특정 기간 동안 롤링을 유지합니다.

인자:

- GCUBE_TOTAL: 사용할 큐브의 총 수
- m[]: 롤링할 이미지를 나타내는 배열
- duration: 롤링을 유지할 시간 (초 단위)
- cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

move_pingpong_robot

설명: 특정 거리만큼 로봇을 이동시키는 함수입니다. 기어 휠 유형과 비기어 휠 유형에 따라 각기 다른 이동 경로를 가집니다.

인자:

- gear_flag: 기어 유형을 나타내는 플래그 (0: 비기어, 1: 기어된 유형, 2: 기어된 유형)
- distance: 이동할 거리 (센티미터 단위)
- cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

rotate_pingpong_robot

설명: 로봇을 특정 각도만큼 회전시키는 함수입니다. 기어 휠 유형과 비기어 휠 유형에 따라 회전 경로가 다릅니다.

인자:

- gear_flag: 기어 유형을 나타내는 플래그 (0: 비기어, 1: 기어된 유형, 2: 기어된 유형)
- angle: 회전할 각도 (도 단위)
- cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

pen_up

설명: 펜을 올리는 함수로, 특정 큐브의 모터를 제어하여 펜을 들어올리는 역할을 합니다.

인자: cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

pen_down

설명: 펜을 내리는 함수로, 특정 큐브의 모터를 제어하여 펜을 내리는 역할을 합니다.

인자: cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

lever_up

설명: 레버를 올리는 함수로, 특정 큐브의 모터를 제어하여 레버를 들어올리는 역할을 합니다.

인자: cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

lever_down

설명: 레버를 내리는 함수로, 특정 큐브의 모터를 제어하여 레버를 내리는 역할을 합니다.

인자: cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

gripper_open

설명: 그리퍼를 열기 위한 함수로, 특정 큐브의 모터를 제어하여 그리퍼를 엽니다.

인자: cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)

gripper_close

설명: 그리퍼를 닫기 위한 함수로, 특정 큐브의 모터를 제어하여 그리퍼를 닫습니다.

인자: cubeSerial: 통신에 사용되는 시리얼 객체 (Stream 클래스)